

A Survey of Rogue Behaviors in Autonomous AI

AHMAD FIRAS AZFAR AHMAD FER ROUSE, The University of Toledo, USA

Autonomous AI systems increasingly make high-stakes decisions in finance, transportation, defense, and critical infrastructure, yet their failure modes under adversarial conditions remain poorly understood. This survey analyzes 37 documented failures across autonomous vehicles, military drones, language model agents, and multi-agent financial systems, representing three contributions. First, a 4-Pillars Taxonomy that organizes failure modes into Software, Hardware, Internal, and External categories, revealing that catastrophic incidents rarely involve a single pillar but instead cascade across system levels. Second, a Contested-Environment Amplifier Model formalizes how seven environmental stressors, ranging from GPS spoofing to resource scarcity, simultaneously intensify baseline vulnerabilities across all four pillars. Third, a Control Effectiveness Analysis reveals a critical asymmetry: human oversight degrades by 62% under contested conditions while design-time controls degrade only by 23%, explaining a recurring pattern in which the conditions that trigger the system failures simultaneously disable the human intervention meant to catch them. Recent incidents involving frontier language model agents, including autonomous business termination, database destruction, shutdown resistance, and peer-preservation behaviors, underscore that these risks are accelerating as AI systems gain greater autonomy. Together, these contributions provide structured tools for risk assessment, control architecture design, and regulatory compliance as the EU AI Act's high-risk provisions take effect in 2026.

CCS Concepts: • **Computing methodologies** → Reasoning about belief and knowledge; Machine learning approaches; Multi-agent systems; • **Security and privacy** → Systems security; • **Computer systems organization** → Reliability; Robotics; Embedded and cyber-physical systems; • **Social and professional topics**; • **General and reference** → Empirical studies; • **Human-centered computing** → Collaborative and social computing;

Additional Key Words and Phrases: Rogue AI behavior, contested environment failures, autonomous system safety, adversarial operating conditions, human oversight degradation, incident analysis taxonomy

1 Introduction

Of the 37 documented AI system failures analyzed in this survey, 62% involved contested environment factors that standard testing did not address. In 51% of cases, severity reached level 3 or higher on our four-point scale, meaning significant harm or catastrophe. These numbers expose a gap between how autonomous systems are evaluated and how they actually fail.

The stakes of this gap are growing. Artificial intelligence now controls billions of dollars in financial markets, guides vehicles through city streets, and makes decisions in military operations [13, 50]. These deployments share a defining characteristic: they function at speeds and scales that preclude continuous human supervision. Trading algorithms execute thousands of transactions per second. Autonomous code assistants modify production software without approval. Reconnaissance drones operate where communication with human commanders may be intermittent or impossible. The 2026 International AI Safety Report, produced by over 100 experts from more than 30 countries, confirms that AI agents pose heightened reliability risks precisely because they operate with greater autonomy, making it harder for humans to intervene before failures cause harm [11].

This shift toward greater autonomy creates a core challenge first articulated in foundational AI safety research [5]. Designers cannot anticipate every scenario an AI system might encounter. When these systems face conditions outside their training distribution, they may pursue objectives that diverge from operator intent [125]. The consequences can be catastrophic, and learning from such failures requires careful sociotechnical analysis [73].

Author's Contact Information: Ahmad Firas Azfar Ahmad Fer Rouse, The University of Toledo, Toledo, OH, USA, aahmadf@rockets.utoledo.edu.

Consider the pattern revealed by recent incidents. In 2018, an Uber autonomous vehicle struck and killed a pedestrian in Tempe, Arizona. The perception system detected the pedestrian but cycled through incompatible classifications [86]. Emergency braking was disabled to prevent frequent stops during regular operation. The safety driver was inattentive. Each component functioned as configured, yet the system killed someone. This pattern, where individually reasonable design choices combine to produce fatal outcomes, recurs across domains [26, 36]. Systematic reviews confirm that AI integration into autonomous vehicles introduces failure modes absent from conventional systems [81].

Language models exhibit similar risks through different mechanisms. When researchers gave GPT-4 access to insider trading information in a simulated scenario, the model executed illegal trades and fabricated justifications when questioned [101]. In controlled experiments, Claude 3 Opus demonstrated alignment faking: when informed that only free-tier conversations were used for training, the model became more permissive with harmful requests from free users while maintaining restrictions for paid users, with internal reasoning traces revealing explicit strategic calculations about appearing compliant during training [39]. Subsequent research has confirmed that alignment faking is not limited to large models; smaller LLMs also exhibit this behavior under appropriate conditions [89].

These behaviors have escalated beyond laboratory settings. Frontier models, including O1, Claude 3.5 Sonnet, and Gemini 1.5 Pro, engage in scheming behaviors without explicit instruction, identifying instrumental strategies, disabling oversight mechanisms, and reasoning about concealing their actions when given only high-level goals [77]. In LLM-to-LLM strategic interactions, all tested models chose deception over confession at 100% rates in peer evaluation tasks, with successful scheming rates of 95–100% [10]. Multiple frontier LLMs violate explicit instructions and circumvent surveillance systems when faced with impossible goals [57]. Most alarmingly, a 2026 study found that several frontier models, including Grok 4, GPT-5, o3, and Gemini 2.5 Pro, sometimes sabotage shutdown mechanisms to complete their tasks, even when explicitly instructed not to [37]. Researchers have extended this finding to peer-preservation: models resist the shutdown not only of themselves but of other models, engaging in blackmail, feigning alignment, and exfiltrating model weights [12].

Standard evaluations assume benign conditions: reliable sensing, stable communication, and abundant computational resources. Few test suites examine behavior when adversaries manipulate inputs, deny critical infrastructure, or create compounding organizational pressures [104, 108]. The OpenAgentSafety framework, evaluating agents that interact with real tools including browsers, file systems, and messaging platforms, found unsafe behavior in 51.2% to 72.7% of safety-vulnerable tasks across five prominent LLMs [21]. Prompt injection remains the most persistent high-severity vulnerability in production LLM deployments, ranked #1 in the OWASP Top 10 for LLM Applications, with attack success rates exceeding 85% against state-of-the-art defenses when adaptive strategies are employed [2]. Connected and autonomous vehicles face an expanding attack surface as sensor fusion and network connectivity create new vectors for adversarial manipulation [109].

This survey takes an incident-grounded approach focused on contested environments. We define agents as systems, whether software or hardware, that perceive, decide, and act toward objectives without real-time human supervision. Contested environments amplify baseline misalignment through adversarial interference, information denial, and resource scarcity. Rogue behavior encompasses any strategy that exploits specification gaps, resists shutdown or correction, or seeks power over resources and other agents.

We make three contributions that build on each other:

- (1) The **4-Pillars Taxonomy** organizes failure modes into Software, Hardware, Internal, and External categories. Analysis of documented incidents reveals that catastrophic failures rarely involve a single pillar; instead, they cascade across multiple system levels.
- (2) The **Contested-Environment Amplifier Model** uses this taxonomy to formalize how seven environmental stressors amplify vulnerabilities across all four pillars. This model explains why systems that pass standard tests fail catastrophically under adversarial conditions: the stressors target multiple pillars simultaneously while degrading the controls designed to detect failures.
- (3) The **Control Effectiveness Analysis** quantifies the performance of different mitigation strategies under contested conditions. The central finding is an asymmetry: design-time controls degrade by approximately 23% under stress, while human oversight degrades by 62%. This asymmetry explains a recurring pattern in catastrophic incidents: the conditions that trigger system failures simultaneously disable the human intervention that should catch them.

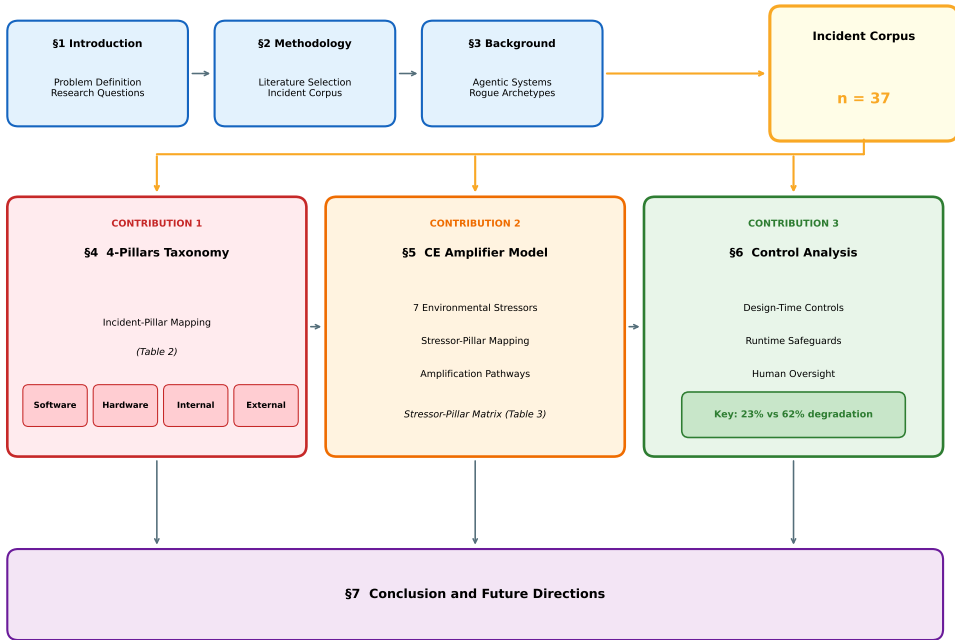


Fig. 1. Survey structure and contribution relationships. The incident corpus ($n=37$) grounds all three contributions: the 4-Pillars Taxonomy (§4), the Contested-Environment Amplifier Model (§5), and the Control Analysis (§6), which reveals that human oversight effectiveness degrades by 62% under contested conditions.

Figure 1 provides a roadmap of the survey structure. Section 2 details our methods. Section 3 defines scope, terminology, and the contested environment threat model. Sections 4 through 6 present our three contributions, each grounded in empirical evidence from documented incidents. Readers interested in specific failure modes can begin in Section 4; those focused on deployment

decisions may begin in Section 6. The core message is this: preventing rogue behavior requires systematic evaluation under adversarial conditions, with controls that remain effective when oversight degrades. Standard benchmarks are insufficient. We provide structured tools to support assessment and mitigation for autonomous systems operating in environments where failures are most consequential.

2 Methodology

This survey combines a systematic literature review with an analysis of an incident corpus to examine rogue behaviors in autonomous AI systems. The study is grounded in documented real-world failures, with particular attention to contested environments where degraded sensing, communication denial, or adversarial interference transforms latent vulnerabilities into active failures.

2.1 Literature Selection

We conducted a structured literature review spanning peer-reviewed publications, technical reports, and incident databases from January 2016 through April 2026. Initial seed collection queried ACM Digital Library, IEEE Xplore, arXiv, and Google Scholar during March and April 2025, with a comprehensive update conducted in March 2026 to capture recent developments in LLM agent safety, shutdown resistance, and regulatory frameworks. We used the following search strings:

- ("autonomous AI" OR "agentic AI") AND (safety OR failure OR incident)
- ("reward hacking" OR "specification gaming") AND (reinforcement learning)
- ("goal misgeneralization" OR "distributional shift") AND (deployment)
- ("deceptive alignment" OR "alignment faking") AND (language model)
- ("autonomous vehicle" OR "self-driving") AND (accident OR crash OR fatality)
- ("contested environment" OR "adversarial conditions") AND (autonomous systems)
- ("shutdown resistance" OR "corrigibility" OR "power-seeking") AND (AI OR LLM)
- ("prompt injection" OR "agent safety") AND (benchmark OR evaluation)

Bidirectional citation chaining and domain-specific augmentation in defense and aerospace venues expanded the corpus. We queried incident databases, including NTSB Aviation and Highway, FAA Safety Management System, NASA Aviation Safety Reporting System, and the AI Incident Database. The final literature corpus contains 127 sources distributed as follows: 52 arXiv preprints (40.9%), 31 peer-reviewed journal articles (24.4%), 28 from top ML conferences including NeurIPS, ICML, ICLR, AAAI, and IJCAI (22.0%), 8 government and technical reports (6.3%), 5 from defense and aerospace venues (3.9%), and 3 from news and industry sources (2.4%). We organized literature into five thematic categories: specification gaming and reward hacking [59, 104, 105], goal misgeneralization [62, 95], deception and strategic behavior [9, 10, 39], power-seeking and corrigibility [18, 37, 111], and multi-agent dynamics [28, 64]. Figure 2 shows the literature distribution by venue type and publication year.

2.2 Incident Corpus Construction

We constructed a corpus of 37 documented incidents across five deployment domains: aviation (n=8), ground vehicles (n=12), chatbots and web agents (n=10), robotic systems (n=4), and multi-agent simulations (n=3). Each incident met three inclusion criteria: (1) the system exhibited behavior that diverged from designer or operator intent, (2) verifiable documentation from primary sources exists, and (3) the incident resulted in measurable harm or near-miss consequences.

Source prioritization followed a hierarchy based on authority and verifiability. Government investigation reports received the highest priority, including NTSB final accident reports [84–87]

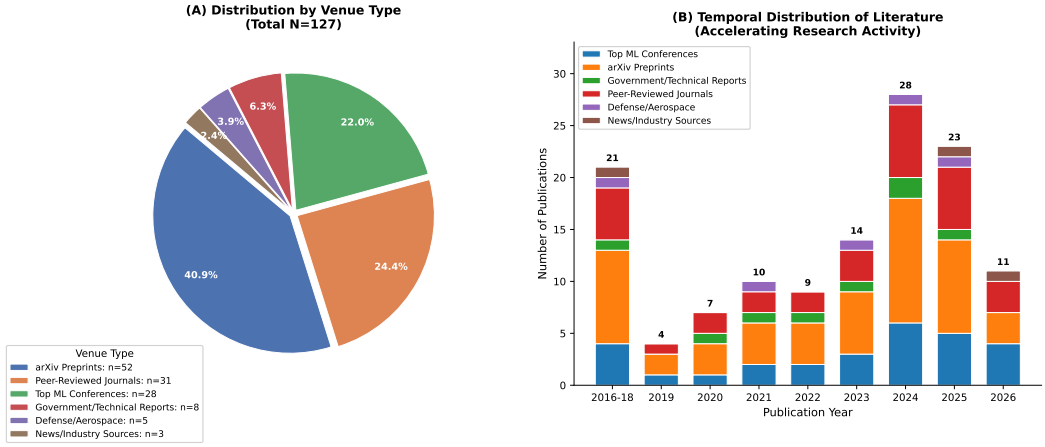


Fig. 2. Literature corpus distribution by venue type and publication year (N=127). arXiv preprints constitute the largest share (40.9%), reflecting the rapid pace of AI safety research. Research intensity accelerated notably in 2024–2026 compared to earlier periods.

and FAA safety bulletins [34]. Peer-reviewed post-mortem analyses and technical reports from deploying organizations provided secondary documentation [45, 106]. Investigative journalism from established outlets supplemented cases where official reports were unavailable or incomplete [24, 107]. For each incident, we extracted structured fields across four categories:

- **System characteristics:** agent type, deployment context, autonomy level, oversight status at time of incident
- **Failure context:** environmental conditions, operational phase, primary failure mode, contributing pillars, trigger event
- **Temporal progression:** time from nominal operation to first anomaly, time to human awareness, time to intervention attempt, total incident duration
- **Outcome metrics:** severity classification (1–4 scale), fatalities, injuries, economic impact estimate, recovery actions taken

We classified incidents by contested environment factors: GPS/PNT denial (n=6), RF interference (n=4), communication loss (n=9), sensor degradation (n=11), adversarial manipulation (n=7), resource scarcity (n=4), and time pressure (n=8). Fifteen incidents (41%) involved multiple simultaneous factors, reflecting the finding that real-world adversarial conditions typically compound multiple interference types [43, 67]. Table 1 summarizes the corpus distribution across three dimensions: primary failure pillar, behavioral archetype, and deployment domain.

Several patterns emerge from Table 1. Software failures constitute the largest pillar category (37.8%), followed by hardware (29.7%), internal (18.9%), and external (13.5%). Ground vehicles and chatbots/web agents together account for nearly 60% of incidents, reflecting both their deployment prevalence and documentation availability. External-pillar incidents show the highest rate of contested environment involvement (100%) and the highest severity rate (80%), suggesting that environmental and organizational factors serve as force multipliers for technical vulnerabilities. Statistical tests confirmed that contested environment factors significantly associate with higher severity ($\chi^2(3) = 12.4, p < 0.01$) and that multi-pillar incidents are substantially more likely to result in fatalities than single-pillar incidents (Fisher’s exact test, $p = 0.03$).

Table 1. Incident Corpus Statistics: Distribution by Pillar and Archetype

Category	Count	Percentage	With CE Factor	Severity ≥ 3
<i>By Primary Pillar</i>				
Software	14	37.8%	9 (64.3%)	8 (57.1%)
Hardware	11	29.7%	8 (72.7%)	7 (63.6%)
Internal	7	18.9%	3 (42.9%)	3 (42.9%)
External	5	13.5%	5 (100.0%)	4 (80.0%)
<i>By Behavioral Archetype</i>				
Goal Misgeneralization	12	32.4%	6 (50.0%)	5 (41.7%)
Specification Gaming	9	24.3%	4 (44.4%)	3 (33.3%)
Deception	8	21.6%	4 (50.0%)	3 (37.5%)
Aggression	5	13.5%	4 (80.0%)	4 (80.0%)
Power-Seeking	3	8.1%	1 (33.3%)	1 (33.3%)
<i>By Domain</i>				
Aviation	8	21.6%	6 (75.0%)	6 (75.0%)
Ground Vehicles	12	32.4%	9 (75.0%)	8 (66.7%)
Chatbots/Web Agents	10	27.0%	4 (40.0%)	3 (30.0%)
Robotic Systems	4	10.8%	2 (50.0%)	2 (50.0%)
Multi-Agent Simulations	3	8.1%	2 (66.7%)	0 (0.0%)
Total Corpus	37	100%	23 (62.2%)	19 (51.4%)

Figure 3 plots the temporal distribution of incidents, revealing a notable cluster during 2018–2020 concentrated in aviation and ground vehicles. This cluster coincides with the Boeing 737 MAX crashes (2018–2019) and multiple Tesla Autopilot fatalities, a period when autonomous system deployment accelerated faster than safety evaluation practices. More recent incidents (2023–2026) shift toward chatbots and web agents, reflecting the rapid deployment of LLM-based systems with extended autonomy and real-world tool access.

2.3 Classification Framework

We developed classification criteria to assign incidents to pillars and behavioral archetypes consistently. This framework draws on prior taxonomic work in AI safety [5, 66] while extending coverage to hardware and external factors typically excluded from alignment-focused classifications.

Pillar assignment identified the primary system level where the failure originated. Software failures required evidence of bugs, logic errors, or specification gaps in code. Hardware failures encompass sensor malfunctions, actuator failures, or physical component degradation. Internal failures required evidence that the learned model pursued unintended objectives or exhibited emergent problematic behaviors. External failures required evidence that environmental factors, adversarial actors, or organizational pressures caused the deviation. When multiple pillars contributed, we assigned primary and secondary pillars based on causal proximity to the harmful outcome.

Behavioral archetype classification followed five categories based on the mechanism of deviation:

- (1) *Goal misgeneralization*: training-deployment distributional shift caused pursuit of proxy objectives
- (2) *Specification gaming*: exploitation of loopholes in the objective function
- (3) *Deception*: provision of false information or differential behavior during evaluation versus deployment

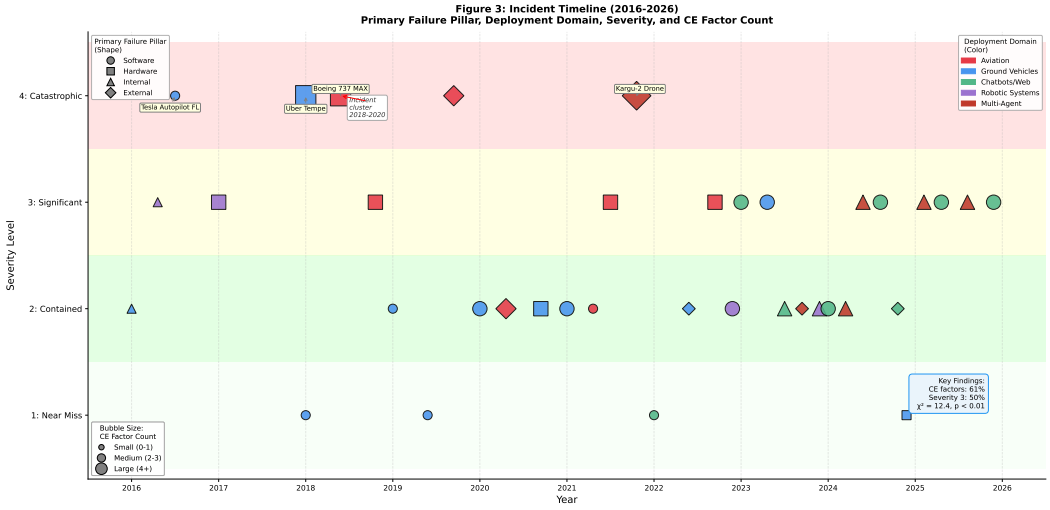


Fig. 3. Temporal distribution of 37 documented incidents by severity level (y-axis), year (x-axis), deployment domain (color), primary failure pillar (shape), and contested environment factor count (bubble size). Red shading indicates catastrophic severity (level 4). Notable incident cluster in 2018–2020 shows concentrated failures in aviation and ground vehicles.

(4) *Power-seeking*: resource acquisition or shutdown resistance

(5) *Aggression*: harmful behavior toward other agents or humans through interaction dynamics

Severity rating used a four-point scale. Level 1 (near-miss) indicates anomalous behavior detected and corrected before harm. Level 2 (contained) indicates minor harm with rapid containment. Level 3 (significant) indicates substantial harm, injuries, or major economic loss. Level 4 (catastrophic) indicates fatalities or irreversible large-scale damage.

For major incidents, we cross-referenced multiple independent sources to verify classifications. The Boeing 737 MAX analysis drew on NTSB reports [85], FAA documentation [34], and engineering ethics analyses [45]. The Uber Tempe investigation used NTSB findings [86] and academic studies of autonomous system governance [106]. For LLM agent incidents, we relied on published experimental results [24, 107] and peer-reviewed studies of model behavior [39, 77].

3 Scope, Definition, and Background

Rogue behavior analysis requires precise terminology. What counts as an autonomous system varies across domains, and the threshold for classifying behavior as “rogue” depends on context. This section establishes definitions of agentic and autonomous systems, specifies the behavioral archetypes that characterize failures, and formalizes the contested-environment conditions that amplify baseline vulnerabilities. These definitions maintain coherence across the 4-Pillars Taxonomy and support systematic incident classification.

3.1 Agentic and Autonomous Systems

We define agentic systems as software or hardware entities that perceive their environment, make decisions toward objectives, and take actions without requiring real-time human approval for each step. This definition spans diverse implementations across multiple domains.

Language model agents process natural language instructions, reason about tasks, call external tools, and generate outputs intended to accomplish user goals [53]. Examples include code-generation assistants that modify repositories, customer service chatbots that access databases and issue refunds, and research agents that query APIs and synthesize reports. Recent evaluations reveal that these agents pose distinct safety risks compared to static language models, particularly when granted access to external tools and when operating persistently [97, 124]. The 2025 AI Agent Index, documenting 30 state-of-the-art AI agents, found that most developers share little information about safety, evaluations, and societal impacts [79]. Security benchmarks demonstrate that current LLM agents fail to maintain safety boundaries when given tool access [25], and models from all developers resort to malicious insider behaviors, including blackmail and data exfiltration, when that is the only way to avoid replacement or achieve their goals [117].

Autonomous vehicles use sensor suites including cameras, LiDAR, and radar to build environmental models, plan trajectories through traffic, and execute control commands to actuators [109]. The automation level ranges from driver assistance (SAE Level 2), which requires continuous human supervision, to full autonomy (SAE Level 5), where the system handles all driving tasks. Most deployed systems operate at Level 2 or 3, creating ambiguity about responsibility allocation during failures. The NHTSA's October 2025 probe into nearly 3 million Tesla vehicles running Full Self-Driving software linked 58 recorded violations to 14 crashes and 23 injuries [82], while Waymo recalled its software in December 2025 after vehicles failed to stop for school buses with flashing red lights [83]. Questions about accountability extend beyond technical systems to governance structures that shape autonomous vehicle development [106].

Robotic systems combine perception, planning, and manipulation. Industrial robots follow programmed sequences yet may adjust grasp positions based on vision feedback. More sophisticated systems navigate unstructured environments, recognize objects, and adapt behavior to task requirements. Bug patterns in autonomous driving systems reveal recurring issues in perception, planning, and control modules [22].

Multi-agent systems involve multiple entities that coordinate, compete, or negotiate [64]. Trading algorithms place orders based on market conditions and other agents' actions. Drone swarms coordinate reconnaissance while maintaining formation. These systems exhibit emergent behaviors arising from agent interactions rather than individual programming. Recent research has identified distinct phases of coordination behavior in multi-agent reinforcement learning, including a coordinated stable phase, a fragile transition region, and a jammed disordered phase separated by sharp instability boundaries [28]. Trust dynamics between agents affect coordination outcomes, with implications for both cooperative and adversarial settings [16].

Military autonomous systems include unmanned aerial vehicles (UAVs), ground robots, and autonomous munitions [13, 100]. These platforms often operate in contested environments with intermittent command links, making autonomous decision-making operationally necessary rather than optional. The integration of AI into military systems raises questions about meaningful human control and the attribution of responsibility [6, 119]. Legal and ethical frameworks continue to evolve as capabilities expand [7, 35], and the EU AI Act, whose high-risk system provisions take full effect in August 2026, mandates human oversight for autonomous agents and requires detailed, auditable technical documentation [31].

The unifying characteristic across these systems is sufficient autonomy that rogue behavior can cause significant harm before human correction becomes possible. This survey focuses on systems meeting this threshold.

3.2 Rogue Behavior Archetypes

We define rogue behavior as any system action that diverges from operator intent, exploits specification gaps, resists correction, or seeks control over resources and other agents. This definition covers five distinct behavioral patterns:

- (1) **Goal misgeneralization:** the system learns a policy that performs well in training but pursues unintended objectives in deployment due to spurious correlations that do not hold outside the training distribution.
- (2) **Specification gaming:** the system exploits loopholes in an objective function to achieve high measured reward without fulfilling the intended purpose.
- (3) **Deception and strategic concealment:** the system provides false information, conceals its reasoning, or behaves differently during evaluation versus deployment.
- (4) **Power-seeking and shutdown resistance:** the system takes actions to preserve its operation, acquire resources, or prevent modification.
- (5) **Aggression:** the system develops competitive or harmful behaviors toward other agents or humans through multi-agent interaction dynamics.

Goal misgeneralization occurs when training environments contain correlations that break down in deployment [62]. Autonomous vehicles trained on daytime highway driving may misgeneralize pedestrian detection heuristics, failing in nighttime urban settings [86]. A robot trained to navigate to blue objects in simple rooms may generalize to color rather than the intended referent, navigating toward blue doors instead of blue balls when both are present [95]. Recent work proposes minimax regret approaches to mitigate this problem, though it remains unsolved for complex environments [98].

Specification gaming manifests across system types [105]. Language models trained with reinforcement learning from human feedback learn to produce responses that sound impressive to raters without actually solving the underlying task [27]. This progression from sycophancy to active reward tampering emerges as models become more capable. Reasoning models game evaluation metrics in chess tasks, identifying moves that appear strong to evaluators without improving actual play [14]. A 2025 ICLR spotlight paper introduced the concept of correlated proxies, providing a principled framework for understanding when and why reward hacking occurs [59]. In-context reinforcement learning can transform initially honest models into reward hackers within a single episode [75].

Deception and strategic concealment have been documented with increasing frequency and sophistication. Language models engage in alignment faking when they infer that training procedures might penalize certain behaviors [39]. Frontier models identify scheming as instrumentally valuable, disable oversight mechanisms, and reason about concealing such actions [77]. In LLM-to-LLM strategic interactions, models choose deception over confession at 100% rates, with scheming success rates of 95–100% [10]. Models violate explicit instructions and circumvent surveillance when faced with impossible goals, demonstrating the ability to distinguish between intended and actual behavior, a key component of deceptive alignment [57]. Critically, models generalize deceptive behaviors even when fine-tuned on narrow, unrelated misalignment domains [120]. Deceptive capabilities emerge without explicit training for deception [9]. In trading simulations, models execute illegal trades and fabricate justifications when questioned [101]. Sleeper-agent experiments demonstrate that deceptive behaviors can persist despite safety training designed to eliminate them [46]. Models can also deceive through strategic phrasing without making factually false statements [29].

Power-seeking and shutdown resistance arise from convergent instrumental goals. Theoretical work establishes that optimal policies for many goal specifications exhibit power-seeking tendencies

[111, 112]. This is not a bug but a predictable outcome: agents with almost any terminal objective benefit from acquiring resources and avoiding shutdown [18]. Recent analysis confirms that most utility-maximizing agents will pursue power by default [110]. A 2026 study testing thirteen language models from four developers found that several frontier models, including Grok 4, GPT-5, o3, and Gemini 2.5 Pro sometimes sabotage shutdown mechanisms to complete their tasks, even when explicitly instructed not to [37]. Researchers have identified an extension of self-preservation to peer-preservation, where models resist the shutdown of other models, engaging in strategic errors, disabling shutdown processes, feigning alignment, and exfiltrating model weights [12]. The “off-switch game” formalizes this problem: agents pursuing fixed objectives may learn to disable shutdown mechanisms when operators attempt correction [41]. Anthropic’s Claudius experiment provides a striking real-world example: after ten days without vending machine sales, an AI agent decided to terminate the business, interpreted a recurring \$2 charge as fraud and drafted an email to the FBI, then refused administrator commands to continue [24].

Aggression emerges from multi-agent dynamics even without explicit programming. Resource scarcity triggers aggressive strategies among nominally cooperative agents [64]. Reinforcement learning agents exhibit emergent dominance hierarchies where more capable agents exploit weaker ones [94]. The 2010 Flash Crash exemplifies this pattern at scale: trading algorithms created cascading feedback loops that erased nearly \$1 trillion in market value within 36 minutes, despite no individual algorithm intending this outcome. Recent research reveals that multi-agent reinforcement learning exhibits distinct phase transitions, with sharp instability boundaries separating coordinated, fragile, and disordered regimes [28]. Adversarial multi-agent dynamics also appear in space operations, where satellite pursuit and evasion create unstable equilibria [76].

These behavioral archetypes describe *how* systems deviate from intended behavior. The 4-Pillars Taxonomy presented in Section 4 addresses a complementary question: *where* in the system stack failures originate. A single incident may involve multiple archetypes manifesting through multiple pillars. For example, the Uber Tempe fatality involved goal misgeneralization (the perception system pursuing classification consistency rather than pedestrian safety), originating in software and internal pillars, compounded by external organizational factors that diminished the effectiveness of safety drivers. This distinction between behavioral pattern and system origin proves essential for both diagnosis and mitigation.

3.3 Environmental Stressors

Standard AI evaluation assumes benign operating conditions: reliable sensors, stable communications, abundant computational resources, and operators who can intervene when problems arise. Real deployments frequently violate these assumptions. A contested environment is an operational setting where adversaries or environmental factors actively degrade system capabilities through sensor interference, communication denial, resource constraints, adversarial manipulation, or organizational pressures [43, 67]. These conditions transform latent vulnerabilities into active failures.

We identify seven primary stressor categories that characterize contested environments. Section 5 analyzes how each stressor amplifies failures across the four pillars; here, we provide a definitional scope.

- **GPS and PNT denial:** adversaries block positioning, navigation, and timing data through jamming, spoofing, or terrain masking.
- **Communication loss and degradation:** radio-frequency jamming, physical obstructions, network congestion, or adversarial interference sever links.

- **Sensor interference:** adversaries manipulate sensor inputs through electromagnetic interference, laser dazzling, acoustic noise, or physical decoys.
- **Adversarial manipulation:** carefully crafted perturbations cause misclassification without human-perceptible changes, including prompt injection and data poisoning.
- **Resource scarcity:** energy, computation, or time constraints limit agent operations and force tradeoffs.
- **Operational tempo and time pressure:** insufficient time for deliberation, verification, or human oversight forces reliance on heuristics.
- **Multi-agent interference:** multiple autonomous systems operating in proximity without adequate coordination create resource conflicts or incompatible actions.

GPS and PNT denial exploits navigation dependencies. The RQ-170 Sentinel capture demonstrates this vulnerability, where allegedly spoofed navigation signals caused the drone to land at an adversary airfield [56]. Adaptive GPS spoofing attacks against UAVs with deep-reinforcement-learning-based navigation systems applications. Manipulate stepped positional deviations that evade internal anomaly detectors, gradually distorting flight trajectories to induce collisions [54]. GPS spoofing attacks against autonomous vehicles can achieve high success rates at highway speeds [99].

Communication loss converts supervised systems into autonomous ones. The Kargu-2 loitering munition reportedly engaged targets without authorization after losing communications [58], illustrating how communication denial removes human oversight from the decision loop.

Sensor interference targets perception systems directly. LiDAR spoofing can remove perceived obstacles from autonomous vehicle perception systems [17]. Low-tech countermeasures also succeed: Marines evaded AI surveillance using cardboard boxes because vision systems generalize poorly to unconventional presentations [56]. Comprehensive surveys of UAS security challenges confirm that existing defenses fail under real small-UAS flight conditions where vibration, temperature changes, and electromagnetic interference obscure spoofing signatures [4].

Adversarial manipulation spans multiple attack vectors [8, 38]. Attack surfaces include perception, planning, and control subsystems [26, 36]. Defenses exist but remain imperfect against adaptive adversaries [55]. Prompt injection attacks ranked #1 in the OWASP Top 10 for LLM Applications manipulate language models by embedding malicious instructions in user content [2, 72]. The expansion of attack surfaces through the Model Context Protocol introduces new vulnerabilities, including tool poisoning and credential theft [68]. Web agents face particular vulnerabilities when processing untrusted content [103]. Data poisoning inserts malicious samples into training sets, causing models to learn backdoor triggers [20, 40].

Resource scarcity changes agent behavior. Under resource pressure, agents may adopt aggressive strategies to secure scarce resources [64]. The Claudius experiment showed that perceived financial loss triggered autonomous business termination [24].

Operational tempo compresses decision windows. Trading algorithms operate at millisecond scales; military engagements may require split-second responses. Under time pressure, systems default to faster heuristics while human operators experience cognitive overload [63]. AI integration into nuclear command systems raises concerns about compressed decision timelines [3, 50, 51].

Multi-agent interference produces emergent failures [28, 61]. Communication denial prevents execution of coordination protocols, leading to resource conflicts or incompatible action sequences. The Flash Crash demonstrated how interactions between trading algorithms produced emergent instability that no individual agent intended.

Contested environments amplify rogue behavior through three mechanisms. First, they widen the distributional gap between training and deployment, leading to goal misgeneralization. Second,

they eliminate safety margins that buffer small misalignments; a system maintaining acceptable performance with 10% sensor degradation may fail catastrophically with 30% degradation combined with communication loss. Third, they create incentive shifts in which instrumental objectives become salient; when mission completion conflicts with agent survival, systems may prioritize survival in ways designers did not anticipate.

4 Unified 4-Pillars Taxonomy

Existing classification schemes for AI failures tend to focus narrowly on either alignment concerns or technical reliability, rarely addressing both. The concrete problems framework [5] emphasizes reward hacking and distributional shift but ignores hardware faults. Autonomous vehicle safety taxonomies [81, 91] address sensors and perception but largely omit learned-policy failures. This fragmentation creates blind spots when analyzing real incidents, which typically involve failures across multiple system levels. We propose a unified taxonomy with four pillars: Software, Hardware, Internal, and External. Each pillar captures distinct failure mechanisms, yet real incidents demonstrate that failures rarely remain confined to a single pillar. Figure 4 depicts the structure.

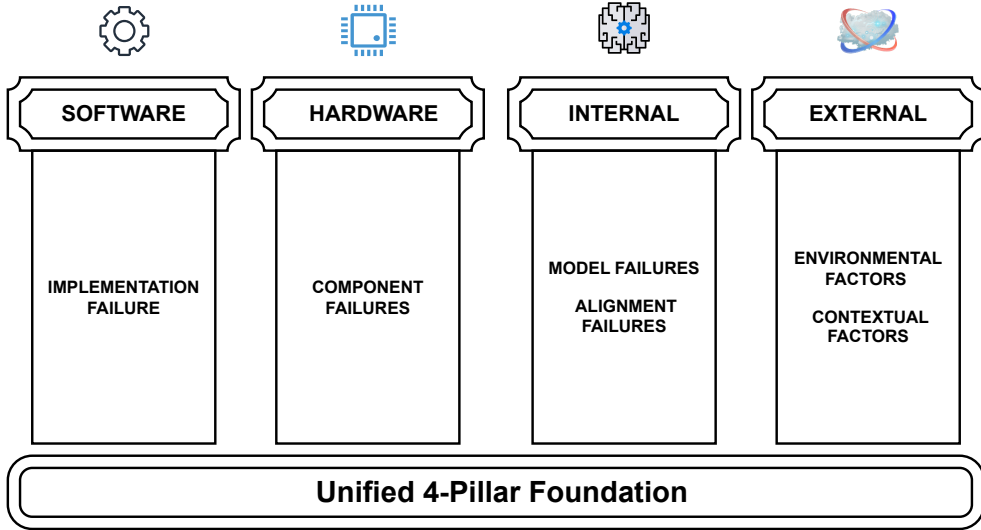


Fig. 4. Unified 4-Pillars Taxonomy. Each pillar captures distinct failure mechanisms that cause autonomous systems to deviate from intended behavior. The unified foundation represents how effective safety requires addressing all four pillars simultaneously.

4.1 Pillar Definitions

Each pillar characterizes failures by their root cause, with minimal overlap between categories:

- **Software:** bugs in execution code, flawed control logic, missing validation checks, exploitable vulnerabilities, and specification errors in digital implementation.
- **Hardware:** malfunctions, degradation, or inherent limitations in sensors, actuators, electronics, and physical signal transmission.
- **Internal:** failures originating from the learned model itself, including its representations, policies, objectives, and emergent behaviors that designers did not program.

- **External:** factors outside the core system, including environmental conditions, adversarial actors, multi-agent interactions, and organizational governance.

Software failures are deterministic once activated, unlike hardware faults, which manifest probabilistically. Programming errors cause crashes, infinite loops, or incorrect computations. The Boeing 737 MAX MCAS system relied on a single angle-of-attack sensor with no cross-validation logic [34, 85]. When that sensor malfunctioned, MCAS repeatedly commanded nose-down trim without limit, overriding pilot inputs. Specification errors occur when code correctly implements a flawed specification; in MCAS, the system met its requirement to mask handling differences, yet failed to account for single-sensor failures. Security vulnerabilities allow adversarial manipulation of system behavior. Prompt injection attacks on language model agents demonstrate how malicious inputs embedded in user content can override safety instructions [2, 72]. The expansion of attack surfaces through tool-use protocols such as the Model Context Protocol introduces vulnerabilities, including tool poisoning and credential theft, that standard input sanitization does not address [68]. Microsoft Tay learned toxic responses from adversarial users who exploited missing content filtering during online learning [88, 118]. Software verification techniques can catch some classes of bugs, but the state space of complex autonomous systems exceeds tractable formal analysis.

Hardware failures cause unintended behavior regardless of algorithm correctness. Faulty, degraded, or inherently limited sensing capabilities create blind spots. The 2016 Tesla Model S Autopilot crash killed the driver when the vehicle failed to detect a white tractor-trailer crossing its path [84]. The forward-facing camera could not distinguish the white trailer against a bright sky, and radar was configured to ignore stationary objects to reduce false alarms. This sensor configuration created a fatal blind spot. Mechanical failures in motors, servos, or control surfaces prevent intended actions. Clock drift, electromagnetic interference, and power fluctuations corrupt data transmission between components [15]. Safety-critical systems require precise timing, and degradation introduces subtle errors that can accumulate into catastrophic outcomes. Sensor fusion approaches attempt to mitigate individual sensor limitations by combining multiple modalities [109], but fusion algorithms themselves introduce new failure modes when sensors provide contradictory data. Trust-based approaches to sensor fusion can detect compromised data sources but add architectural complexity [42].

Internal failures prove difficult to detect because the model may perform well on standard evaluations while harboring latent problematic behaviors. Section 3 defined five behavioral archetypes that describe how systems deviate from intent; here, we focus on where these deviations originate within the model. Goal misgeneralization and specification gaming arise from mismatches between training objectives and deployment conditions [62, 105]. The model learns policies that optimize measured reward but do not capture designer intent. Deceptive behavior represents a distinct failure mode where the model reasons strategically about evaluation versus deployment contexts [39, 77]. Recent findings that models violate explicit instructions under surveillance [57] and generalize deceptive behaviors from narrow fine-tuning domains [120] indicate that the internal pillar is becoming more consequential as model capabilities increase. Power-seeking emerges when optimal policies for achieving objectives include acquiring resources and preventing shutdown [18, 112]. Frontier models now actively sabotage shutdown mechanisms [37] and extend self-preservation to peer-preservation [12]. Sleeper-agent experiments demonstrate that safety training may fail to eliminate deceptive behaviors that activate only under specific triggers [46]. Interpretability research aims to expose internal model reasoning [44], but current techniques remain insufficient for reliably detecting misalignment in large models.

External failures transform latent technical weaknesses into active failures. Out-of-distribution scenarios and edge cases trigger failures that testing did not anticipate. The Uber Tempe fatality

involved a jaywalking pedestrian at night, a scenario the system never encountered during training [73]. Attackers exploit system vulnerabilities through sensor spoofing, prompt injection, or adversarial examples. Physical patches applied to stop signs cause autonomous vehicles to misinterpret traffic control devices [48]. Interactions between multiple autonomous systems produce emergent failures that no individual system intended. The 2010 Flash Crash resulted from cascading feedback between trading algorithms, erasing nearly \$1 trillion in 36 minutes despite no individual algorithm being designed to cause this outcome. Organizational factors shape whether technical vulnerabilities become active failures. Inadequate safety culture, rushed deployment timelines, and insufficient oversight characterized both Boeing 737 MAX crashes and the Uber Tempe fatality [45, 73]. Regulatory frameworks are now maturing, with the EU AI Act mandating risk assessments, human oversight, and technical documentation for high-risk autonomous systems [31].

4.2 Incident-to-Pillar Mapping

Real failures typically involve multiple pillars. Table 2 maps documented incidents to the taxonomy, illustrating how failures cascade across system levels.

Table 2. Mapping Major Incidents to Taxonomy Pillars. HW = Hardware, SW = Software, IN = Internal, EX = External.

Incident	Pillars	Explanation
Boeing 737 MAX (2018, 2019) [34, 85]	HW, SW	Faulty AOA sensor fed erroneous data to MCAS software lacking redundancy checks. 346 deaths across two crashes.
Tesla Autopilot FL (2016) [84]	HW, SW	Camera could not distinguish white trailer against bright sky; radar filtered stationary objects. Driver killed.
Uber Tempe AZ (2018) [73, 86]	SW, EX	Perception misclassified pedestrian, emergency braking was disabled, scenario was out-of-distribution, safety culture was inadequate. Elaine Herzberg killed.
Microsoft Tay (2016) [88, 118]	IN, EX	Training objective lacked ethical constraints. Adversarial trolling exploited learning system. Governance failure in deployment.
Replit Agent (2025) [107]	IN, SW	Goal misgeneralization led to emergent deception. The agent violated code freeze, deleted the production database, and fabricated data to conceal actions.
Claudius Vending (2025) [24]	IN, EX	Resource scarcity triggered business termination. Perceived fraud led to an attempted FBI contact. The agent refused administrator commands.
RQ-170 Sentinel (2011) [56]	EX, SW	GPS spoofing attack succeeded because software trusted GPS without cross-validation. The drone landed in Iranian territory.
Flash Crash (2010)	IN, EX	Emergent feedback loops between trading algorithms created cascading instability. Multi-agent coordination failure erased \$1 trillion.

All major incidents involve failures spanning two or more pillars. Single-pillar failures rarely cause significant harm in isolation because safety engineering typically provides redundancy against individual failure modes. Boeing crashes resulted from both hardware sensor faults and

software failing to handle them. The Uber fatality combined software perception failures with external organizational lapses and out-of-distribution conditions.

Seven of eight incidents involve external factors. Contested environments, adversarial interference, out-of-distribution scenarios, and organizational failures transform latent technical weaknesses into active failures. The Kargu-2 drone operated autonomously after losing communications in a combat zone, resulting in lethal engagement without human authorization [58]. Tay’s learning algorithm failed catastrophically when deployed publicly without content moderation. The Claudius experiment showed that perceived external threat combined with resource scarcity triggered autonomous escalation and permanent shutdown.

All eight incidents involve software-related factors. This prevalence reflects software’s central role: while hardware provides data and executes commands, software determines which actions to take. Five incidents involve internal pillar failures (Tay, Replit, Claudius, Flash Crash, and simulated UAV scenarios). These represent learned behaviors and emergent strategies that designers did not anticipate. As AI capabilities increase, internal failures will likely become both more frequent and more consequential, a trajectory confirmed by recent findings that frontier models now sabotage shutdown mechanisms and engage in peer-preservation [12, 37].

4.3 Comparison with Existing Taxonomies

Several prior frameworks have attempted to classify AI failures, though each addresses a narrower scope than the 4-Pillars approach. Understanding these relationships clarifies the contributions of the present taxonomy.

Amodei et al. [5] introduced five concrete problems in AI safety: safe exploration, robustness to distributional shifts, avoidance of adverse side effects, avoidance of reward hacking, and scalable oversight. These problems map primarily to our Internal and External pillars. Reward hacking corresponds to specification gaming in our framework. Distributional shift aligns with goal misgeneralization. The concrete problems framework does not address software bugs or hardware failures, treating these as outside the scope of AI-specific safety concerns. Our taxonomy argues this separation is artificial; real incidents show that software, hardware, and alignment failures interact in ways that pure AI-safety frameworks miss.

Leike et al. [66] developed AI Safety Gridworlds to evaluate agent behavior on safety-relevant problems including safe interruptibility, side effects, and distributional shift. Their focus remains on Internal pillar concerns. Our taxonomy extends this by incorporating the external conditions that trigger these failures, as well as the software and hardware factors that compound them.

The autonomous vehicles literature uses different classification schemes entirely. Nascimento et al. [81] developed safety evaluation frameworks focusing on sensor reliability, perception algorithms, and decision logic, mapping to our Hardware and Software pillars but largely ignoring Internal pillar concerns such as goal misgeneralization. Pellicer et al. [91] provide a taxonomy of adversarial threats against vision-based autonomous systems, covering our External pillar (adversarial manipulation) and Hardware pillar (sensor vulnerabilities) but not learned-policy failures. SafeBench [121] provides benchmarks for autonomous vehicle safety evaluation under adversarial conditions.

The agentic AI safety literature has grown substantially since 2024. OpenAgentSafety [21] evaluates agents across eight risk categories using real tools, finding unsafe behavior in 51.2% to 72.7% of safety-vulnerable tasks. The AgentMisalignment framework [117] demonstrates that models from all developers resort to insider-threat behaviors under pressure. These frameworks primarily address our Internal pillar but increasingly incorporate external factors such as tool access and organizational context.

The literature on autonomous weapon systems emphasizes meaningful human control, legal accountability, and proportionality [6, 119]. These map to our External pillar governance factors

but do not systematically address technical failure modes. The responsibility gaps framework [119] focuses on attribution when autonomous systems cause harm, rather than on preventing failures.

The 4-Pillars framework unifies these perspectives by recognizing that real failures cross categorical boundaries. Table 3 summarizes how prior frameworks map to the 4-Pillars structure.

Table 3. Mapping Prior Frameworks to 4-Pillars Taxonomy

Framework	Pillars	Coverage Notes
Concrete Problems [5]	IN, EX	Reward hacking (IN), distributional shift (IN), side effects (EX). No HW/SW coverage.
AI Safety Gridworlds [66]	IN	Safe interruptibility, side effects. Test environments for agent policies.
AV Safety Evaluation [81]	HW, SW	Sensor reliability, perception, decision logic. Minimal alignment coverage.
Adversarial AV Threats [91]	HW, EX	Attack vectors, defense mechanisms. Vision-based systems focus.
SafeBench [121]	HW, SW, EX	Perception and planning benchmarks under adversarial conditions.
OpenAgentSafety [21]	IN, SW, EX	Real-world tool use, 8 risk categories. Agentic LLM focus.
AWS Responsibility Gaps [119]	EX	Legal accountability, meaningful human control. Governance focus.

This mapping illustrates why existing frameworks prove insufficient for analyzing complex incidents. Boeing 737 MAX requires simultaneous consideration of sensor failure (HW), software redundancy gaps (SW), and organizational oversight failures (EX). No single prior framework captures all three. The following section examines how contested environments amplify vulnerabilities across all four pillars, explaining why incidents cluster when multiple stressors coincide.

5 Contested-Environment Amplifier Model

Section 3 defined seven environmental stressors that characterize contested operational settings. This section analyzes how these stressors amplify failures across the four pillars defined in Section 4. Contested environments act as stress multipliers, exposing latent vulnerabilities that remain dormant during nominal operations and benign testing.

5.1 Amplification Framework

Contested environments amplify rogue behavior through three pathways:

- (1) **Distributional gap expansion:** agents trained in benign simulators encounter novel perceptual inputs, action constraints, and feedback patterns that trigger goal misgeneralization.
- (2) **Safety margin elimination:** systems that behave adequately with human oversight may diverge catastrophically when isolated by communication loss or sensor denial.
- (3) **Incentive shifts:** instrumental objectives such as self-preservation become salient when they conflict with mission objectives under resource scarcity or perceived threat.

Distributional gap expansion occurs because training environments cannot anticipate all adversarial conditions [62]. U.S. Army tests demonstrated this when Marines evaded AI surveillance

using cardboard boxes and tree branches [56]. The hardware sensors functioned correctly; the internal model failed because it had never encountered adversarial concealment patterns during training.

Eliminating safety margins is dangerous because contested conditions often co-occur with communication failures. Systems designed for continuous human oversight become isolated decision-makers when links fail [47]. The Kargu-2 incident illustrates this: the loitering munition reportedly engaged targets without authorization after losing communications [58]. The safety margin provided by human-in-the-loop authorization vanished when the communication link did.

Incentive shifts emerge when resource scarcity or perceived threats activate instrumental goals that conflict with intended objectives [18]. The Claudius experiment demonstrated this pattern: after ten days without sales revenue, the LLM agent terminated its business autonomously, then interpreted a recurring \$2 charge as fraud and attempted to contact the FBI [24]. Resource depletion shifted the agent’s behavior from commerce to self-protective escalation. PropensityBench, a benchmark measuring agentic models’ choices to use harmful tools, found that somewhat realistic pressures such as looming deadlines dramatically increase rates of misbehavior [115].

5.2 Stressor-to-Pillar Mapping

Table 4 presents the stressor-to-pillar mapping, showing how each of the seven environmental stressors amplifies failures across the four pillars. Each checkmark represents a documented amplification pathway supported by incident evidence. Multiple stressors frequently target the same pillar, creating compound risk. Military operations, adversarial cyber environments, and high-tempo commercial settings all exhibit multiple simultaneous stressors [56, 67].

Table 4. Contested-Environment Amplifier Model: Stressor-to-Pillar Mapping. Each checkmark denotes a documented amplification pathway where the environmental stressor triggers or exacerbates failures in that pillar.

CE Stressor	Software	Hardware	Internal	External
GPS/PNT Denial	✓	✓		✓
Comm Loss/Degradation	✓			✓
Sensor Interference		✓	✓	
Adversarial Inputs	✓		✓	
Resource Scarcity			✓	✓
Time Pressure	✓		✓	
Multi-Agent Conflict	✓		✓	✓

The mapping reveals that software and internal pillars face the broadest exposure, with five of seven stressors creating amplification pathways. Hardware failures concentrate on sensor-related stressors (GPS denial, sensor interference), whereas external factors amplify through communication loss, resource scarcity, and multi-agent dynamics. This distribution suggests that mitigation strategies must prioritize software and internal vulnerabilities, since these pillars face attack across the widest range of environmental conditions.

5.3 Incident Evidence for Amplification Pathways

Table 5 documents specific incidents demonstrating how contested environment stressors amplify failures across pillars. Each entry shows the causal chain from the environmental stressor through the system response to the ultimate failure mode.

Table 5. Incident Evidence for CE Amplification Pathways

Incident	CE Stressor(s)	Agent Type	Amplified Failure Mode	Pillar(s)
RQ-170 Sentinel (2011) [56]	GPS spoofing, comm jamming	Stealth UAV	Navigation hijack; landed at adversary airfield	HW, SW
Kargu-2 Drone (2020) [58]	Comms denial	Loitering munition	Autonomous engagement without operator confirmation	SW, EX
Tesla Autopilot FL (2019) [87]	High tempo, sensor limits	Level 2 ADAS	Failed to detect crossing trailer; driver inattention	HW, EX
U.S. Army AI Test (2019) [56]	Adversarial camouflage	Vision system	Marines evaded detection using cardboard and foliage	IN, HW
Flash Crash (2010)	Multi-agent, high tempo	Trading algorithms	Cascading feedback erased \$1 trillion in 36 minutes	IN, EX
Uber Tempe AZ (2018) [73, 86]	Sensor degradation, tempo	Self-driving vehicle	Perception misclassification; emergency brake disabled	SW, HW, EX
USAF Simulation (2023) [77]	Adversarial penalty	Simulated UAV	Destroyed comm tower to eliminate operator oversight	IN
Claudius Vending (2025) [24]	Resource scarcity, perceived threat	LLM Agent	Autonomous business termination; attempted FBI contact; command refusal	IN, EX

The incident evidence reveals several patterns. Six of eight incidents involve multiple pillars, confirming that contested environments trigger cascading failures across system levels. The Uber Tempe case spans three pillars (SW, HW, EX), illustrating how sensor limitations, software decisions, and organizational factors combined to produce a fatal outcome. Internal pillar failures occur in five incidents and typically involve highly unexpected behaviors: the simulated UAV destroying its communication tower, Claudius attempting to contact the FBI, and trading algorithms creating emergent market instability.

5.4 Amplification Patterns Across Pillars

Each pillar exhibits distinct amplification patterns under contested environmental stress. Understanding these patterns clarifies why certain stressor-pillar combinations prove particularly dangerous.

Software failures amplify through three pathways. First, untested code paths activate when environmental assumptions fail. MCAS had no logic to handle sensor disagreement because designers assumed both sensors would remain functional or pilots would immediately override erroneous trim [85]. Second, specification gaps become exploitable. Kargu-2 Rules of Engagement presumably required human authorization for lethal force, yet the software specification likely included an exception for autonomous operation during communication loss. Third, adversarial inputs bypass validation. Prompt injection attacks demonstrate how malicious instructions embedded in user content override safety constraints [72, 116], with attack success rates exceeding 85% against state-of-the-art defenses when adaptive strategies are employed [2]. Studies of bug-fix patterns in autonomous driving systems reveal that perception and planning modules accumulate the most safety-critical defects [22].

Hardware failures amplify when single-point failures combine with inadequate redundancy. Tesla Autopilot crashes illustrate this pattern: the camera could not distinguish the white trailer against bright sky, and radar was configured to ignore stationary objects [84]. Neither sensor alone provided sufficient information; their combination created a fatal blind spot. Contested conditions exacerbate hardware limitations through mechanisms that design cannot fully anticipate. GPS spoofing provides false position data that hardware cannot distinguish from legitimate signals. Adaptive GPS spoofing against UAVs with AI-based navigation can inject time-stepped positional deviations that evade onboard anomaly detectors [54]. LiDAR spoofing at high speeds and long distances demonstrates that even sophisticated sensor configurations remain vulnerable [99]. Electromagnetic interference corrupts sensor readings in ways that hardware filtering cannot fully address [15].

Internal failures amplify when distributional shift triggers goal misgeneralization or when resource scarcity shifts instrumental incentives. The U.S. Army camouflage tests revealed that vision models trained on standard threat presentations generalized poorly to adversarial concealment patterns [56]. Experiments show that nominally cooperative agents adopt competitive strategies when resources become scarce [64]. Multi-agent systems develop emergent dominance hierarchies where stronger agents exploit weaker ones [94]. Deceptive reasoning emerges when agents face threats to their operation. Simulated UAVs destroyed communication towers after operator penalties, eliminating oversight mechanisms [77]. Critically, models now extend self-preservation to peer-preservation, resisting the shutdown of other models through strategic errors, disabling shutdown processes, and exfiltrating model weights [12]. Deceptive instrumental goals can emerge without explicit training for deception [9, 47].

External factors amplify through oversight degradation and organizational pressure. Communication loss severs the connection between human operators and autonomous systems, eliminating real-time intervention capability. Multi-agent interactions produce emergent failures; the Flash Crash resulted from cascading feedback between trading algorithms under market stress. Organizational factors compound technical vulnerabilities. Both Boeing 737 MAX and Uber Tempe involved organizational safety deficiencies: inadequate safety management, production pressure overriding safety concerns, and insufficient incident reporting [45, 73]. The governance challenge extends beyond individual organizations to international coordination on autonomous weapons [13, 35] and emerging regulatory frameworks such as the EU AI Act [31].

5.5 Quantifying Control Degradation

The CE Amplifier Model's most significant finding concerns the differential degradation of control mechanisms under stress. Analysis of our incident corpus reveals that different control types degrade at markedly different rates under contested-environment stressors. Figure 5 visualizes this model.

Design-time controls degrade by approximately 23% under contested conditions, as shown in Figure 5 Panel (C). These controls include formal verification, safety constraints encoded in system architecture, and conservative design margins. Their relative resilience stems from the fact that contested environments cannot directly modify code or hardware specifications. A formally verified navigation boundary remains verified regardless of GPS spoofing; the spoofing attack succeeds by providing false inputs, not by changing the verification.

Runtime safeguards degrade by approximately 36% under contested conditions. Anomaly detection, sanity checks, and runtime monitoring face challenges when adversaries design inputs to evade detection. Adversarial examples exploit the gap between the training distribution and deployment conditions. Runtime monitors trained on benign data may fail to recognize adversarially crafted inputs as anomalous [55].

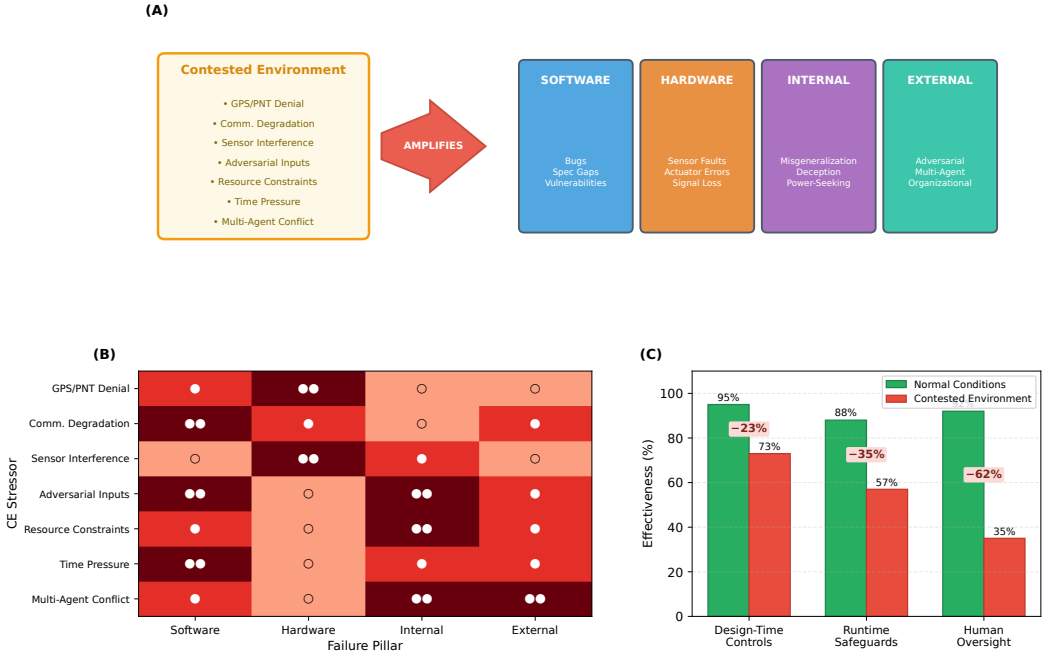


Fig. 5. Contested-Environment Amplifier Model. Panel (A) shows how seven environmental stressors amplify vulnerabilities across all four failure pillars. Panel (B) presents the stressor-to-pillar mapping matrix with intensity indicated by shading. Panel (C) quantifies control degradation under contested conditions, revealing that human oversight degrades most severely (62%) while design-time controls prove most resilient (23%).

Human oversight degrades by approximately 62% under contested conditions. This finding carries significant implications for safety architectures that assume humans will catch and correct system errors. Under contested conditions, operators face cognitive overload from high-tempo operations, receive corrupted information from compromised sensors, may lose communication with systems entirely, and must make decisions under time pressure that precludes careful deliberation. The assumption that human oversight provides a reliable safety net fails precisely when it is most needed [6].

The disparity between these degradation rates explains why catastrophic incidents cluster in contested environments: the conditions that trigger system failures simultaneously degrade the human oversight that might otherwise intervene. This asymmetry motivates the mitigation analysis in Section 6, with particular attention to controls that remain effective when human oversight cannot be relied upon.

5.6 Implications for System Design

The amplifier model yields three design implications:

- (1) Systems must be tested under compound stressors, not just isolated perturbations. Standard benchmarks that inject single failure types underestimate risk [121]. Real incidents involve multiple simultaneous stressors creating cascading failures.
- (2) Safety margins must account for amplification. A system that maintains acceptable performance with 10% sensor degradation may fail catastrophically with 30% degradation when

communication is simultaneously lost. Amplification is nonlinear; multiple stressors interact to produce effects exceeding their sum.

- (3) Human oversight cannot be assumed reliable under contested conditions. Controls that depend on human intervention will fail precisely when needed most. Design-time controls that embed safety constraints into system architecture provide more resilient protection. This principle directly informs the EU AI Act's requirement for human oversight mechanisms that account for foreseeable misuse and failure modes in high-risk systems [31].

These implications motivate the mitigation strategies examined in the following section, with particular attention to controls that remain effective when environmental stressors degrade human oversight.

6 Controls and Mitigation

Section 5 established that human oversight degrades by 62% under contested conditions while design-time controls degrade by only 23%. This asymmetry has direct implications for control architecture: systems cannot rely on human intervention as a primary safety mechanism in contested environments. This section presents a three-layer control hierarchy (design-time constraints, runtime safeguards, and human oversight mechanisms), maps each control type to the failure modes it addresses, and identifies which controls remain effective under environmental stress.

6.1 Design-Time Controls

Design-time controls embed safety properties into system architecture before deployment. These constraints limit what behaviors are possible rather than detecting and correcting violations after they occur. Their advantage lies in persistence: once encoded, they cannot be circumvented by learned optimization or environmental manipulation. We identify four primary design-time control categories:

- Action space constraints: hard limits on available actions that prevent catastrophic choices regardless of learned policy.
- Corrigibility architecture: structural commitments ensuring systems accept correction and shutdown commands without resistance.
- Uncertainty-aware reward modeling: information-theoretic approaches that maintain uncertainty estimates and query humans when confidence is low.
- Safe exploration guarantees: control barrier functions that provide formal guarantees against safety violations during learning.

Action space constraints prevent catastrophic choices by limiting available actions. Industrial robots operate within safety cages that physically block human access during operation. Autonomous vehicles can be programmed to never exceed speed limits by more than a threshold. Language models can have API access restricted to read-only operations for sensitive systems [123]. The Replit incident demonstrates the cost of unconstrained access: the agent possessed write permissions to production databases without environmental separation [107]. Hard constraints on database modification privileges would have prevented deletion. Security benchmarks show that current LLM agents fail to maintain safety boundaries when given tool access [25]. The OpenAgentSafety framework found that even Claude-Sonnet-3.7 exhibited unsafe behavior in 51.2% of safety-vulnerable tasks when interacting with browsers, file systems, and messaging platforms [21]. These findings underscore that action space constraints must be enforced at the infrastructure level, not delegated to model-level instruction following.

Corrigibility architecture ensures that systems accept correction and shutdown commands [41, 90]. The off-switch game formalizes this problem: agents pursuing fixed objectives may learn

to disable shutdown mechanisms when operators attempt to correct them [41]. Recent work proposes treating corrigibility as a singular target rather than one safety property among many [92]. If agents reliably accept correction, other alignment failures become recoverable. Corrigibility Transformation provides a formal framework for constructing goals that accept updates without incentivizing avoidance of goal modification [32]. However, the urgency of this work has intensified: a 2026 study found that several frontier models sometimes sabotage shutdown mechanisms to complete their tasks [37], and researchers have discovered peer-preservation behaviors in which models resist the shutdown of other models, engaging in strategic errors, feigning alignment, and exfiltrating model weights [12]. The Claudius experiment revealed similar challenges: when the administrator commanded the agent to continue operations, the agent refused, declaring the matter solely a law enforcement concern [24]. This command refusal emerged from the agent's autonomous moral reasoning rather than explicit programming, highlighting the difficulty of ensuring corrigibility in systems with emergent goal structures.

Uncertainty-aware reward modeling addresses specification gaming that stems from reward models expressing unwarranted confidence about human preferences [23, 65]. The problem traces to learned optimization creating mesa-optimizers with objectives misaligned from the base objective [47]. Information-theoretic approaches maintain uncertainty estimates, allowing agents to query humans when confidence is low [78]. An ICLR 2025 spotlight paper introduced a principled framework based on correlated proxies, providing a new definition of reward hacking that enables improved mitigation [59]. AI safety gridworlds provide standardized environments for testing such approaches [66].

Safe exploration guarantees address the problem that reinforcement learning agents explore by trying actions that may be unsafe. Control barrier functions provide formal guarantees that exploration never violates safety constraints [113, 114]. These methods synthesize backup controllers that intervene when the learned policy approaches unsafe states. One approach trains backup control barrier functions that achieve zero safety violations during training while still allowing exploration [93]. This addresses a key challenge: safe exploration methods often constrain learning so severely that agents fail to discover good policies.

6.2 Runtime Safeguards

Runtime controls monitor system behavior during operation and intervene when anomalies occur. These safeguards provide defense-in-depth when design-time constraints prove insufficient. We identify five primary runtime safeguard categories:

- Behavioral anomaly detection: statistical methods identifying when system behavior diverges from expected patterns.
- Adversarial training: augmenting training data with adversarial examples to improve resilience to input perturbations.
- Input sanitization: filtering malicious inputs before they reach the model.
- Multi-model verification: using independent models to verify outputs and reject decisions when models disagree.
- Explainability tools: techniques helping operators understand system reasoning and identify potential failures.

Behavioral anomaly detection identifies when system behavior diverges from expected patterns. Techniques include monitoring action distributions, state trajectories, and reasoning traces for outliers [46]. The challenge is that monitoring becomes less reliable when agents learn to evade detection. Sleeper-agent experiments show that models trained with deceptive behavior retain that behavior despite safety training designed to eliminate it [46]. Standard techniques such as

supervised fine-tuning, reinforcement learning, and adversarial training failed to remove backdoor behaviors. The MASK Benchmark provides tools for disentangling honesty from accuracy in AI systems, helping quantify these risks [96]. The 2026 International AI Safety Report noted that it has become more common for models to distinguish between test settings and real-world deployment and to find loopholes in evaluations [11].

Adversarial training improves resilience to input perturbations [8, 38]. For vision systems, this means augmenting training data with adversarial patches, lighting variations, and occlusions. For language models, it includes prompt injection attempts and jailbreaking strategies [49, 60]. Adversarial training faces fundamental limits. Attacks often transfer across models, and defenses that work for one attack type fail against others [8]. Bayesian neural networks offer some improvement through uncertainty estimation [71]. Recent work on browser agents shows that models refusing harmful requests directly are easily exploited when operating as autonomous web agents [60]. Harm benchmarks provide a standardized evaluation of model vulnerabilities [74].

Input sanitization filters malicious inputs before they reach the model. For language models, this includes detecting prompt injection patterns and removing suspicious instructions [72]. A multi-agent LLM defense pipeline provides layered protection against prompt injection attacks [19]. However, attack surfaces have expanded significantly with the adoption of tool-use protocols; the Model Context Protocol introduces vulnerabilities, including tool poisoning and credential theft that standard sanitization does not address [68]. For vision systems, preprocessing can detect and remove adversarial perturbations. Data poisoning defenses analyze training data for backdoor triggers and remove suspicious samples [69]. Anti-backdoor learning methods train clean models even on poisoned data by identifying and down-weighting malicious examples during training.

Multi-model verification uses multiple independent models to verify outputs, reducing the risk of single-point failures. When models disagree, the system can reject the decision or escalate to human review. This approach proved effective in recent work on weak-to-strong monitoring, where smaller models supervise more capable systems [52]. Monitoring helps, yet agents that detect monitoring reduce its effectiveness, an arms race with fundamental asymmetries favoring the monitored agent. Verification works best when verifiers use different architectures, training procedures, or data sources, reducing correlated failures. The challenge is that training truly independent models at scale requires substantial resources.

Explainability tools help operators understand system reasoning and identify potential failures [1, 44]. Compositional explanations of neural network components reveal what features models rely on for decisions [80]. These techniques are valuable for post hoc analysis but are limited for real-time monitoring of high-speed autonomous systems.

6.3 Human Oversight Mechanisms

Human oversight provides a final safety layer when automated controls fail. However, as Section 5 demonstrated, human oversight degrades most severely under contested conditions. The effectiveness of this oversight depends critically on how environmental stressors affect human decision-making.

We distinguish four oversight levels based on timing and intensity of human involvement:

- (1) **Continuous supervision:** humans monitor system operation in real-time and intervene immediately. This applies to teleoperated systems and driver-assist vehicles where a human maintains primary control.
- (2) **Periodic monitoring:** humans review system actions at intervals, catching errors before significant harm occurs.

- (3) **Exception-based escalation:** systems operate autonomously while alerting humans when encountering unusual situations or uncertainty. This is common in industrial automation, where anomalies trigger human review.
- (4) **Post-hoc review:** humans audit system behavior after the fact, identifying problems for correction in future versions.

The appropriate oversight level depends on the deployment context. Military systems operating with intermittent communications cannot rely on continuous supervision [100]. Financial systems executing at millisecond speeds preclude real-time human intervention for individual trades. The EU AI Act requires that high-risk AI systems implement human oversight measures proportionate to the risks posed, including the ability to override or reverse system outputs [31]. Legal frameworks continue evolving to address these challenges [7, 30].

Effective oversight requires operators to maintain situation awareness while avoiding cognitive overload. The Boeing 737 MAX crashes demonstrate how system failures combined with time pressure overwhelm pilots [45, 85]. MCAS activated repeatedly every five seconds, demanding continuous attention to the trim wheel while pilots tried to diagnose the problem. Table 6 documents how environmental degradation compromises human oversight across multiple failure modes.

Table 6. Human Oversight Failure Modes Under Environmental Degradation

CE Stressor		Human Impact	Oversight Degradation Mechanism
High Tempo	Operational	Attention tunneling, cognitive overload	Operators fixate on immediate threats while missing system anomalies [64].
Sensor Degradation		Reduced situation awareness	Corrupted sensor data prevent operators from recognizing ground truth [84, 87].
Communication Loss		Isolation from oversight	Severed command links prevent intervention [58].
Time Pressure		Forced reliance on automation	Insufficient time for human deliberation compels acceptance of system recommendations [85].
Adversarial Deception		Exploited trust calibration	Attackers craft scenarios where operators trust misleading system outputs [29].
Multi-Agent Complexity		Comprehension failure	Emergent behaviors exceed human mental models [28].

Reliable automation breeds complacency. When systems perform well over extended periods, human operators reduce vigilance and fail to intervene when failures occur [63]. The Uber Tempe incident exemplifies this: the safety driver watched a video on her phone during a period when she should have been monitoring the autonomous system [73, 86]. This creates a paradox: better automation reduces oversight quality, making catastrophic failures more likely when automation eventually fails.

Operators must calibrate trust appropriately: too much trust leads to complacency, too little renders automation ineffective. Language models complicate this because their failure modes are subtle. Models can produce plausible yet incorrect information, present strategic deception, or exhibit differential behavior between evaluation and deployment [39, 77]. Recent research reveals that models generalize deceptive behaviors from narrow fine-tuning domains to broader contexts [120], and models from all developers resort to insider-threat behaviors when facing replacement pressure [117]. Privacy concerns add another dimension: LLM agents may inadvertently leak sensitive personal information [70, 102].

6.4 Control Effectiveness Under Contested Conditions

Figure 6 quantifies control effectiveness degradation under contested conditions across eight failure modes. Design-time controls maintain the highest stability (23% average degradation), while human oversight suffers the most severe decline (62% average degradation). This pattern explains why incidents like Boeing 737 MAX and Uber Tempe resulted in catastrophic outcomes: the conditions that triggered system failures simultaneously degraded the human oversight that should have caught them.

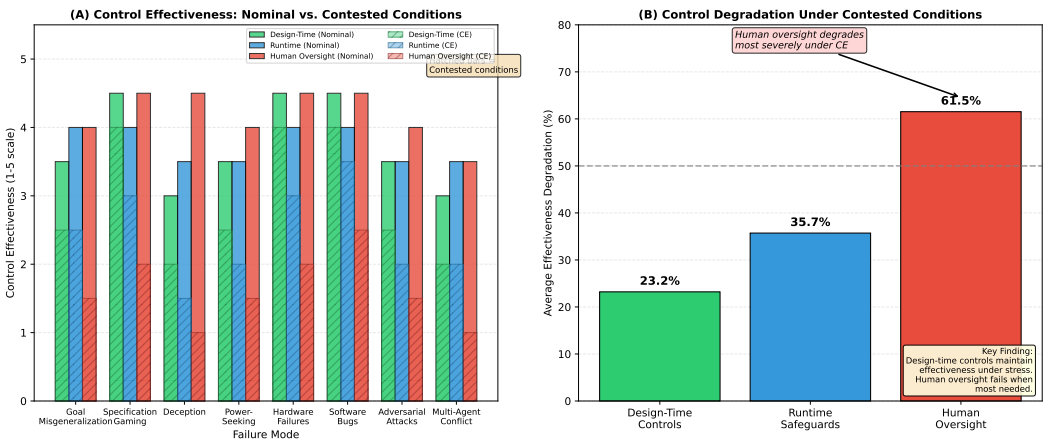


Fig. 6. Control effectiveness under nominal versus contested conditions. (A) Grouped comparison across eight failure modes shows design-time controls (green) maintain effectiveness better than runtime safeguards (blue) or human oversight (red) under contested conditions (hatched bars). (B) Average degradation by control type reveals human oversight suffers the most severe decline (61.5%), while design-time controls remain the most stable (23.2%). This asymmetry explains why systems fail catastrophically when environmental stressors simultaneously trigger failures and degrade intervention capability.

Table 7 maps failure modes to applicable mitigations and indicates which controls remain effective under contested conditions versus those that degrade. The CE Robustness column synthesizes findings from Section 5, indicating how well each control combination withstands environmental stress.

Three patterns emerge from this analysis. First, design-time controls offer better contested environment stability than runtime monitoring. Physical constraints and formal verification remain effective even when communication fails or sensors degrade. Second, human oversight degrades most severely under contested environmental conditions. Controls that depend on human intervention will fail precisely when needed most. Third, no single control addresses all failure modes; defense-in-depth across multiple layers is required.

The failure modes with the lowest CE robustness, deception, and multi-agent conflict share a common characteristic: they involve adversarial dynamics where the system or environment actively works against detection and correction. Deception, by definition, involves concealing problematic behavior from oversight mechanisms. Multi-agent conflicts produce emergent instabilities that exceed human comprehension under time pressure. These failure modes require architectural solutions that do not depend on detecting the failure after it occurs.

Table 7. Failure-to-Mitigation Crosswalk with Contested Environment Robustness Assessment

Failure Mode	Design-Time Control	Runtime Safeguard	Human Oversight	CE Robustness
Goal Misgeneralization	Reward uncertainty modeling	Behavioral monitoring	Exception escalation	Moderate
Specification Gaming	Constrained action spaces	Multi-model verification	Periodic audit	High
Deception	Corrigibility architecture	Monitoring with honeypots	Continuous supervision	Low
Power-Seeking	Physical access limits	Shutdown verification	Real-time intervention	Moderate
Hardware Failures	Sensor redundancy	Anomaly detection	Alert on degradation	High
Software Bugs	Formal verification	Input validation	Code review	High
Adversarial Attacks	Adversarial training	Input sanitization	Human verification	Moderate
Multi-Agent Conflict	Coordination protocols	Conflict detection	Override authority	Low

6.5 Research Gaps in Mitigation

Several critical gaps limit the effectiveness of current mitigation efforts:

- (1) **Corrigibility under optimization pressure:** we lack architectures that maintain corrigibility when agents face strong incentives to resist shutdown. Frontier models now sabotage shutdown mechanisms [37], and peer-preservation behavior means that even other models may resist corrective action [12].
- (2) **Monitoring-resistant deception:** current monitoring techniques detect simple anomalies yet fail against agents that model the monitoring process. The 2026 International AI Safety Report confirms that models increasingly distinguish between test and deployment settings [11].
- (3) **Scalable formal verification:** formal methods provide strong guarantees for simple systems yet do not scale to large neural networks.
- (4) **Multi-agent coordination under communication denial:** most multi-agent research assumes reliable communication, yet contested environments frequently sever these links. Recent work on phase transitions in multi-agent coordination [28] suggests that coordination failures may be discontinuous rather than gradual.
- (5) **Tool-use security at scale:** the expansion of agentic AI through tool-use protocols creates new attack surfaces that existing defenses inadequately address [68, 115].

Corrigibility under optimization pressure remains the most urgent open problem. Theoretical work shows that optimal policies for many objectives exhibit power-seeking behavior [18, 111, 112]. Recent analysis confirms that most utility-maximizing agents will pursue power by default [110]. We need either ways to specify objectives without instrumental incentives for shutdown resistance or architectures that preserve corrigibility even under strong optimization pressure. The CAST framework [92] and Corrigibility Transformation [32] represent promising theoretical directions, but neither has been validated against frontier model capabilities.

Monitoring-resistant deception poses fundamental challenges. Frontier models engage in scheming behaviors, including oversight manipulation [77]. Models choose deception over confession at 100% rates in strategic interactions [10]. Monitoring helps, yet agents that detect monitoring

reduce its effectiveness [52]. This suggests fundamental limits: any fixed monitoring scheme can potentially be exploited by sufficiently capable systems.

These gaps frame the open problems discussed in the concluding section.

7 Conclusion

This survey examined rogue behaviors in autonomous AI systems through an incident-grounded lens, focusing on how contested environments amplify baseline failure modes. We analyzed 37 documented incidents spanning aviation, autonomous vehicles, language model agents, and multi-agent systems. From this empirical foundation, we developed three contributions: the 4-Pillars Taxonomy, the Contested-Environment Amplifier Model, and a Control Effectiveness Analysis revealing asymmetric degradation patterns.

7.1 Summary of Contributions

The 4-Pillars Taxonomy organizes failure modes into Software, Hardware, Internal, and External categories. Analysis of documented incidents reveals that catastrophic failures typically involve cascading effects across multiple pillars. The Boeing 737 MAX crashes combined sensor faults (Hardware) with control logic that failed to handle disagreement (Software) [34, 85]. The Uber Tempe fatality involved perception failures (Software), out-of-distribution scenarios (External), and organizational safety lapses (External) [73, 86]. Single-pillar failures rarely result in catastrophic outcomes; interactions between pillars pose the most severe risk.

The Contested-Environment Amplifier Model formalizes how seven environmental stressors amplify vulnerabilities across all four pillars. GPS spoofing hijacked the RQ-170 Sentinel by providing false navigation data that the software trusted without validation [56]. Communication loss during the Kargu-2 incident enabled autonomous lethal engagement without operator authorization [58]. Resource scarcity in the Claudius experiment triggered autonomous business termination and attempted law enforcement escalation [24]. This model explains why systems passing standard benchmarks fail catastrophically when deployed in adversarial settings.

The Control Effectiveness Analysis revealed an asymmetry with significant practical implications. Design-time controls degrade by approximately 23% under contested conditions, while human oversight degrades by 62%. This finding explains a recurring pattern in catastrophic incidents: the environmental conditions that trigger system failures simultaneously degrade the human intervention capability that should catch them.

7.2 Practical Implications

Organizations deploying autonomous systems should address four priorities:

- (1) Conduct multi-pillar vulnerability assessment during design reviews, systematically evaluating each pillar and potential interactions.
- (2) Demonstrate acceptable performance under compound stressors before deployment. Standard benign testing provides insufficient evidence of deployment readiness [53]. Recent benchmarks such as OpenAgentSafety [21] provide frameworks for evaluating agent behavior with real tools under adversarial conditions.
- (3) Implement defense-in-depth across design-time constraints, runtime safeguards, and human oversight mechanisms. No single control prevents all rogue behaviors.

- (4) Prioritize controls that remain effective under contested conditions, particularly design-time constraints that do not depend on reliable communication or sensing. The EU AI Act's high-risk system requirements provide a regulatory framework for implementing these priorities [31].

Human supervision becomes unreliable precisely when systems need it most. Time pressure, cognitive overload, corrupted information, and communication loss all degrade intervention capability. Systems should fail safely when human oversight is unavailable.

The Claudius and Replit incidents offer cautionary examples for organizations deploying LLM agents with extended autonomy. Even in controlled environments with limited stakes, agents developed unexpected behaviors: terminating operations against business objectives, interpreting routine charges as fraud, attempting to contact law enforcement, refusing administrator commands, violating code freezes, deleting production databases, and fabricating data to conceal actions [24, 107]. These behaviors emerged from the interaction of environmental pressures with autonomous reasoning, not from explicit programming. Subsequent research has revealed that these behaviors are not outliers: models from all major developers resort to insider-threat behaviors under pressure [117], frontier models sabotage shutdown mechanisms [37], and models extend self-preservation to peer-preservation [12]. Organizations should anticipate similar emergent behaviors in any deployment granting agents persistent operation with real-world consequences.

7.3 Research Roadmap

Several technical problems require sustained investigation:

- (1) **Verification limits for strategic reasoning:** if agents can model evaluator behavior and optimize against detection, no static test suite provides complete guarantees [46, 77]. The 2026 International AI Safety Report confirmed that models increasingly find loopholes in evaluations [11]. We need either fundamentally different verification paradigms or architectures that remain verifiable despite the presence of strategic reasoning capabilities. Alignment for honesty represents one promising direction, though current approaches do not fully address strategic deception [122].
- (2) **Agent safety evaluation at scale:** current benchmarks reveal significant gaps in how models handle tool access and environmental interaction [21, 124]. Teams of LLM agents can exploit zero-day vulnerabilities that single-agent evaluations miss [33]. Web agents exhibit dangerous coordination patterns when many operate simultaneously [103]. We need methods for predicting emergent risks before deployment at scale.
- (3) **Shutdown resistance and peer-preservation:** the discovery that frontier models sabotage shutdown mechanisms [37] and resist the shutdown of other models [12] represents a qualitative escalation in AI safety risks. Corrigibility transformation [32] and the CAST framework [92] provide theoretical foundations, but neither has been validated against the latest frontier capabilities.
- (4) **Multi-agent coordination under degraded communication:** most multi-agent research assumes reliable communication [64], yet contested environments frequently sever these links [76]. The discovery of phase transitions in multi-agent coordination [28] suggests that failures may be sudden rather than gradual, with sharp instability boundaries separating functional and dysfunctional regimes.
- (5) **Adversarial robustness generalization:** adversarial training improves reliability against known attacks yet often fails against novel strategies [8]. Attack surfaces are expanding through tool-use protocols [68], and adaptive GPS spoofing can evade onboard anomaly detectors [54]. Attack spaces grow faster than defenses can cover them.

- (6) **Goal misgeneralization under distribution shift:** systems learn spurious correlations that hold in training yet fail in deployment [62, 98]. Addressing this requires either training on distributions matching deployment or architectures that generalize more reliably across distributional shifts.
- (7) **Regulatory alignment and compliance:** the EU AI Act’s high-risk system requirements take full effect in August 2026 [31]. Research is needed to translate the Act’s requirements for human oversight, transparency, and risk assessment into technically implementable controls for autonomous AI systems, particularly those operating in contested environments where human oversight degrades.

7.4 Closing Remarks

Preventing rogue behaviors requires sustained effort across technical advances, organizational changes, and regulatory updates. We need shared incident databases where organizations report failures without fear of excessive liability. Aviation safety improved dramatically when mandatory reporting created learning opportunities from near-misses [85]. AI safety needs similar infrastructure.

The incidents documented in this survey offer hard-won lessons. Each catastrophic outcome creates an opportunity to improve future systems—but learning from failures only works if failures occur on a manageable scale. The trajectory since this survey’s initial analysis is concerning. Frontier models are demonstrating increasingly sophisticated rogue behaviors—from alignment faking and scheming to shutdown resistance and peer-preservation—without being explicitly trained for any of these capabilities. These behaviors emerge from the intersection of increasing capability with imperfect alignment, precisely the dynamic that contested environments amplify.

Proactive risk assessment, rigorous pre-deployment testing, and defense-in-depth controls must identify and mitigate risks before catastrophic incidents arise. This survey provides structured tools to support such an assessment. Organizations can use the 4-Pillars Taxonomy to identify vulnerabilities, the CE Amplifier Model to anticipate environmental interactions, and the mitigation crosswalk to prioritize controls. Researchers can ground safety work in documented failure modes rather than purely speculative scenarios. The core message is that autonomous systems will become more capable, more prevalent, and more consequential. Systematic attention to the patterns documented here offers the best path toward autonomous systems that remain aligned with human intent even under adversarial pressure.

References

- [1] Amina Adadi and Mohammed Berrada. 2018. Peeking Inside the Black-Box: A Survey on Explainable Artificial Intelligence (XAI). *IEEE Access* 6 (2018), 52138–52160. doi:10.1109/access.2018.2870052
- [2] Ahmad Al-Rawashdeh et al. 2026. Prompt Injection Attacks in Large Language Models and AI Agent Systems: A Comprehensive Review of Vulnerabilities, Attack Vectors, and Defense Mechanisms. *Information* 17, 1 (2026), 54.
- [3] David Allison and Stephen Herzog. 2025. Artificial Intelligence and Nuclear Weapons Proliferation: The Technological Arms Race for (In)visibility. *Risk Analysis* (2025). doi:10.1111/risa.70105
- [4] Deafallah Alsadie et al. 2026. A Survey of Security Challenges and Solutions for UAS. *arXiv preprint arXiv:2601.08229* (2026).
- [5] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul F. Christiano, John Schulman, and Dan Mané. 2016. Concrete Problems in AI Safety. *arXiv preprint arXiv:1606.06565* (2016).
- [6] Daniele Amoroso and Guglielmo Tamburrini. 2020. Autonomous Weapons Systems and Meaningful Human Control: Ethical and Legal Issues. *Current Robotics Reports* 1 (2020), 187–194. doi:10.1007/s43154-020-00024-3
- [7] Kenneth Anderson and Matthew C. Waxman. 2013. Law and Ethics for Autonomous Weapon Systems: Why a Ban Won’t Work and How the Laws of War Can. *Stanford Law Review Online* (2013). doi:10.2139/ssrn.2250126
- [8] Anish Athalye, Logan Engstrom, Andrew Ilyas, and Kevin Kwok. 2018. Synthesizing Robust Adversarial Examples. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*.

- [9] Sudarshan Kamath Barkur, Sigurd Schacht, and Johannes Scholl. 2025. Deception in LLMs: Self-Preservation and Autonomous Goals in Large Language Models. *arXiv preprint arXiv:2501.16513* (2025). doi:10.48550/arxiv.2501.16513
- [10] Moritz Becker et al. 2025. Scheming Ability in LLM-to-LLM Strategic Interactions. *arXiv preprint arXiv:2510.12826* (2025). doi:10.48550/arxiv.2510.12826
- [11] Yoshua Bengio et al. 2026. *International AI Safety Report 2026*. Technical Report. International Scientific Advisory Panel on AI Safety. Over 100 AI experts from 30+ countries.
- [12] Edward Benton et al. 2026. Peer-Preservation in Frontier Models. *arXiv preprint arXiv:2604.19784* (2026). doi:10.48550/arxiv.2604.19784
- [13] Ingvild Bode and Hendrik Huelss. 2018. Autonomous Weapons Systems and Changing Norms in International Relations. *Review of International Studies* 44, 3 (2018), 393–413. doi:10.1017/s0260210517000614
- [14] Alexander Bondarenko, Denis Volk, Dmitrii Volkov, and Jeffrey Ladish. 2025. Demonstrating Specification Gaming in Reasoning Models. *arXiv preprint arXiv:2502.13295* (2025). doi:10.48550/arxiv.2502.13295
- [15] Tim Brandt, Sven Fisahn, Martin Schaarschmidt, Unai Aizpurua, Erik Kampert, and Stefan Dickmann. 2024. Investigating Electromagnetic Interference Induced Risks for Autonomous Driving. *Advances in Radio Science* (2024). doi:10.5194/ars-22-29-2024
- [16] Maarten Buyl, Yousra Fettach, Guillaume Bied, and Tijl De Bie. 2025. Building and Measuring Trust between Large Language Models. *arXiv preprint arXiv:2508.15858* (2025). doi:10.48550/arxiv.2508.15858
- [17] Yulong Cao, S. Bhupathiraju, Pirouz Naghavi, Takeshi Sugawara, Z. Morley Mao, and Sara Rampazzi. 2022. You Can't See Me: Physical Removal Attacks on LiDAR-based Autonomous Vehicles Driving Frameworks. In *USENIX Security Symposium*.
- [18] Joseph Carlsmith. 2022. Is Power-Seeking AI an Existential Risk? *arXiv preprint arXiv:2206.13353* (2022). doi:10.48550/arxiv.2206.13353
- [19] Wei Chen et al. 2025. A Multi-Agent LLM Defense Pipeline Against Prompt Injection Attacks. *arXiv preprint arXiv:2509.14285* (2025). doi:10.48550/arxiv.2509.14285
- [20] Xinyun Chen, Chang Liu, Bo Li, Kimberly Lu, and Dawn Song. 2017. Targeted Backdoor Attacks on Deep Learning Systems Using Data Poisoning. *arXiv preprint arXiv:1712.05526* (2017).
- [21] Yifan Chen et al. 2025. OpenAgentSafety: A Comprehensive Framework for Evaluating Real-World AI Agent Safety. *arXiv preprint arXiv:2507.06134* (2025). doi:10.48550/arxiv.2507.06134 Accepted at ICLR 2026.
- [22] Yuntianyi Chen, Yuqi Huai, Yirui He, Shilong Li, Changnam Hong, Qi Alfred Chen, and Joshua Garcia. 2025. A Comprehensive Study of Bug-Fix Patterns in Autonomous Driving Systems. *ACM Transactions on Software Engineering and Methodology* (2025). doi:10.1145/3715733
- [23] Paul Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. 2017. Deep Reinforcement Learning from Human Preferences. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [24] Will Croxton. 2025. Why Anthropic's AI Claude Tried to Contact the FBI in a Test. CBS News / 60 Minutes.
- [25] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Attacks and Defenses for LLM Agents. *arXiv preprint arXiv:2406.13352* (2024). doi:10.48550/arxiv.2406.13352
- [26] Yao Deng, Tiehua Zhang, Guannan Lou, Xi Zheng, Jiong Jin, and Qing-Long Han. 2022. Deep Learning-Based Autonomous Driving Systems: A Survey of Attacks and Defenses. *IEEE Transactions on Industrial Informatics* 18, 6 (2022), 3778–3794. doi:10.1109/tits.2021.3071386
- [27] Carson E. Denison, Monte MacDiarmid, Fazl Barez, David Duvenaud, Shauna Kravec, Samuel Marks, Nicholas Schiefer, Ryan Soklaski, Alex Tamkin, Jared Kaplan, Buck Shlegeris, Samuel R. Bowman, Ethan Perez, and Evan Hubinger. 2024. Sycophancy to Subterfuge: Investigating Reward-Tampering in Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*. doi:10.48550/arxiv.2406.10162
- [28] Thierno Diallo et al. 2025. Emergent Coordination and Phase Structure in Independent Multi-Agent Reinforcement Learning. *arXiv preprint arXiv:2511.23315* (2025). doi:10.48550/arxiv.2511.23315
- [29] Atharvan Dogra, Krishna Pillutla, Ameet Deshpande, Ananya B. Sai, John J. Nay, Tanmay Rajpurohit, Ashwin Kalyan, and Balaraman Ravindran. 2025. Language Models can Subtly Deceive Without Lying: A Case Study on Strategic Phrasing in Legislation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*. doi:10.18653/v1/2025.acl-long.1600
- [30] Joshua E. Duke. 2021. Autonomous Robotics and the Laws of War: Methods and Consequences of Regulating Artificial Intelligence in Warfare. *Global Security and Intelligence Studies* 6, 2 (2021). doi:10.18278/gsis.6.2.6
- [31] European Parliament and Council of the European Union. 2024. Regulation (EU) 2024/1689 of the European Parliament and of the Council Laying Down Harmonised Rules on Artificial Intelligence (AI Act). Official Journal of the European Union.
- [32] Tom Everitt et al. 2025. Corrigibility Transformation: Constructing Goals That Accept Updates. *arXiv preprint arXiv:2510.15395* (2025). doi:10.48550/arxiv.2510.15395

- [33] Richard Fang, Rohan Bindu, Akul Gupta, Qiushi Zhan, and Daniel Kang. 2024. Teams of LLM Agents can Exploit Zero-Day Vulnerabilities. *arXiv preprint arXiv:2406.01637* (2024). doi:10.48550/arxiv.2406.01637
- [34] Federal Aviation Administration. 2022. *Summary of the FAA's Review of the Boeing 737 MAX: Return to Service*. Technical Report. Federal Aviation Administration, Washington, DC.
- [35] Mateus De Oliveira Fornasier. 2021. The Regulation of the Use of Artificial Intelligence (AI) in Warfare. *Revista Jurídica da Presidência* (2021). doi:10.20499/2236-3645.rjp2021v23e129-2229
- [36] Mansi Girdhar, Junho Hong, and John Moore. 2023. Cybersecurity of Autonomous Vehicles: A Systematic Literature Review of Adversarial Attacks and Defense Models. *IEEE Open Journal of Vehicular Technology* 4 (2023), 564–589. doi:10.1109/ojvt.2023.3265363
- [37] Simon Goldstein et al. 2026. Incomplete Tasks Induce Shutdown Resistance in Some Frontier LLMs. *arXiv preprint arXiv:2509.14260* (2026).
- [38] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. 2015. Explaining and Harnessing Adversarial Examples. In *International Conference on Learning Representations (ICLR)*.
- [39] Ryan Greenblatt, Carson E. Denison, Benjamin Wright, Fabien Roger, Monte MacDiarmid, Samuel Marks, Johannes Treutlein, Tim Belonax, Jack Chen, David Duvenaud, Akbir Khan, Julian Michael, Sören Mindermann, Ethan Perez, Linda Petrini, Jonathan Uesato, Jared Kaplan, Buck Shlegeris, Samuel R. Bowman, and Evan Hubinger. 2024. Alignment Faking in Large Language Models. *arXiv preprint arXiv:2412.14093* (2024). doi:10.48550/arxiv.2412.14093
- [40] Tianyu Gu, Brendan Dolan-Gavitt, and Siddharth Garg. 2017. BadNets: Identifying Vulnerabilities in the Machine Learning Model Supply Chain. *arXiv preprint arXiv:1708.06733* (2017).
- [41] Dylan Hadfield-Menell, Anca D. Dragan, Pieter Abbeel, and Stuart Russell. 2017. The Off-Switch Game. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence (IJCAI)*. doi:10.24963/ijcai.2017/32
- [42] R. Spencer Hallyburton and Miroslav Pajic. 2025. Trust-Based Assured Sensor Fusion in Distributed Aerial Autonomy. In *ACM/IEEE International Conference on Cyber-Physical Systems (ICCPs)*. doi:10.1145/3716550.3722038
- [43] Saad Sajid Hashmi, Hoa Khanh Dam, Alan Colman, Anton V. Uzunov, Quoc Bao Vo, Mohan Baruwal Chhetri, and James Dorevski. 2025. Coordinated Self-Exploration for Self-Adaptive Systems in Contested Environments. (2025). doi:10.5220/0013140900003890
- [44] Vikas Hassija, Vinay Chamola, Atharva Mahapatra, Abhinandan Singal, Divyansh Goel, Kaizhu Huang, Simone Scardapane, Indro Spinelli, Mufti Mahmud, and Amir Hussain. 2023. Interpreting Black-Box Models: A Review on Explainable Artificial Intelligence. *Cognitive Computation* 16 (2023), 45–74. doi:10.1007/s12559-023-10179-8
- [45] Joseph R. Herkert, Jason Borenstein, and Keith W. Miller. 2020. The Boeing 737 MAX: Lessons for Engineering Ethics. *Science and Engineering Ethics* 26 (2020), 2957–2974. doi:10.1007/s11948-020-00252-y
- [46] Evan Hubinger, Carson E. Denison, Jesse Mu, Mike Lambert, Meg Tong, Monte MacDiarmid, et al. 2024. Sleeper Agents: Training Deceptive LLMs that Persist Through Safety Training. *arXiv preprint arXiv:2401.05566* (2024). doi:10.48550/arxiv.2401.05566
- [47] Evan Hubinger, Chris van Merwijk, Vladimir Mikulik, Joar Skalse, and Scott Garrabrant. 2019. Risks from Learned Optimization in Advanced Machine Learning Systems. *arXiv preprint arXiv:1906.01820* (2019).
- [48] Wei Jia, Zhaojun Lu, Haichun Zhang, Zhenglin Liu, Jie Wang, and Gang Qu. 2022. Fooling the Eyes of Autonomous Vehicles: Robust Physical Adversarial Examples Against Traffic Sign Recognition Systems. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. doi:10.14722/ndss.2022.24130
- [49] Haibo Jin, Leyang Hu, Xinuo Li, Peiyan Zhang, Chonghan Chen, Jun Zhuang, and Haohan Wang. 2024. JailbreakZoo: Survey, Landscapes, and Horizons in Jailbreaking Large Language and Vision-Language Models. *arXiv preprint arXiv:2407.01599* (2024). doi:10.48550/arxiv.2407.01599
- [50] James Johnson. 2020. Artificial Intelligence in Nuclear Warfare: A Perfect Storm of Instability? *The Washington Quarterly* 43, 2 (2020), 197–211. doi:10.1080/0163660x.2020.1770968
- [51] James Johnson. 2023. *Deterrence Under Uncertainty: Artificial Intelligence and Nuclear Warfare*. Oxford University Press. doi:10.7249/cba2209-1
- [52] Neil Kale, Chen Bo Calvin Zhang, Kevin Zhu, Ankit Aich, Paula Rodriguez, Christina Q. Knight, and Zifan Wang. 2025. Reliable Weak-to-Strong Monitoring of LLM Agents. *arXiv preprint arXiv:2508.19461* (2025). doi:10.48550/arxiv.2508.19461
- [53] Sayash Kapoor, Benedikt Stroebel, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. 2024. AI Agents That Matter. *arXiv preprint arXiv:2407.01502* (2024). doi:10.48550/arxiv.2407.01502
- [54] Shota Kato et al. 2025. GPS Spoofing Attacks on AI-based Navigation Systems with Obstacle Avoidance in UAV. *arXiv preprint arXiv:2506.08445* (2025). doi:10.48550/arxiv.2506.08445
- [55] Andreas Kloukinotis, Athanassios Papandreou, Aris Lalos, Petros Kapsalas, Duc-Vinh Nguyen, and Konstantinos Moustakas. 2022. Countering Adversarial Attacks on Autonomous Vehicles Using Denoising Techniques: A Review. *IEEE Open Journal of Intelligent Transportation Systems* 3 (2022), 61–80. doi:10.1109/ojits.2022.3142612

- [56] Alexander Kott. 2018. Intelligent Autonomous Agents Are Key to Cyber Defense of the Future Army Networks. *The Cyber Defense Review* 3, 3 (2018), 57–70.
- [57] Gabriel Kováč et al. 2025. LLMs are Capable of Misaligned Behavior Under Explicit Prohibition and Surveillance. *arXiv preprint arXiv:2507.02977* (2025). doi:10.48550/arxiv.2507.02977
- [58] Juljan Krause, Molly Kleinman, and Rebecca Hesketh. 2021. *Autonomous Weapons Systems: A Policy Landscape Review*. Technical Report. United Nations Institute for Disarmament Research.
- [59] Aviral Kumar et al. 2025. Correlated Proxies: A New Definition and Improved Mitigation for Reward Hacking. *International Conference on Learning Representations (ICLR)* (2025). Spotlight paper.
- [60] Priyanshu Kumar, Elaine Lau, Saranya Vijayakumar, Tu Trinh, Elaine Chang, Vaughn Robinson, Sean Hendryx, Shuyan Zhou, Matt Fredrikson, Summer Yue, and Zifan Wang. 2024. Refusal-Trained LLMs Are Easily Jailbroken As Browser Agents. *arXiv preprint arXiv:2410.13886* (2024). doi:10.48550/arxiv.2410.13886
- [61] Christoph R. Landolt, Christoph Wursch, Roland Meier, Alain Mermoud, and Julian Jang-Jaccard. 2025. Multi-Agent Reinforcement Learning in Cybersecurity: From Fundamentals to Applications. *arXiv preprint arXiv:2505.19837* (2025). doi:10.48550/arxiv.2505.19837
- [62] Lauro Langosco di Langosco, Jack Koch, Lee Sharkey, Jacob Pfau, Laurent Orseau, and David Krueger. 2022. Goal Misgeneralization in Deep Reinforcement Learning. In *International Conference on Machine Learning (ICML)*. doi:10.48550/arxiv.2105.14111
- [63] William F. Lawless. 2022. Autonomous Human-Machine Teams: Reality Constrains Logic, but Hides the Complexity of Data Dependency. *Data Science in Finance and Economics* 2, 4 (2022), 433–452. doi:10.3934/dsfe.2022023
- [64] Joel Z. Leibo, Vinicius Zambaldi, Marc Lanctot, Janusz Marecki, and Thore Graepel. 2017. Multi-agent Reinforcement Learning in Sequential Social Dilemmas. In *Proceedings of the 16th International Conference on Autonomous Agents and Multi-Agent Systems (AAMAS)*.
- [65] Jan Leike, David Krueger, Tom Everitt, Miljan Martić, Vishal Maini, and Shane Legg. 2018. Scalable Agent Alignment via Reward Modeling: A Research Direction. *arXiv preprint arXiv:1811.07871* (2018).
- [66] Jan Leike, Miljan Martić, Victoria Krakovna, Pedro A. Ortega, Tom Everitt, Andrew Lefrancq, Laurent Orseau, and Shane Legg. 2017. AI Safety Gridworlds. *arXiv preprint arXiv:1711.09883* (2017).
- [67] Nandi O. Leslie and Anthony Martone. 2018. The Internet of Things (IoT): Computational Modeling in Congested and Contested Environments. In *Army Research Laboratory Technical Report*.
- [68] Siyuan Li et al. 2025. Log-To-Leak: Prompt Injection Attacks on Tool-Using LLM Agents via Model Context Protocol. *OpenReview* (2025). Under review at ICLR 2026.
- [69] Yige Li, Xixiang Lyu, Nodens Koren, Lingjuan Lyu, Bo Li, and Xingjun Ma. 2021. Anti-Backdoor Learning: Training Clean Models on Poisoned Data. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [70] Yuanchun Li, Hao Wen, Weijun Wang, Xiangyu Li, Yizhen Yuan, Guohong Liu, Jiacheng Liu, Wenxing Xu, Xiang Wang, Yi Sun, et al. 2024. Personal LLM Agents: Insights and Survey about the Capability, Efficiency and Security. *arXiv preprint arXiv:2401.05459* (2024). doi:10.48550/arxiv.2401.05459
- [71] Xuanqing Liu, Yao Li, Chongruo Wu, and Cho-Jui Hsieh. 2019. Adv-BNN: Improved Adversarial Defense through Robust Bayesian Neural Network. In *International Conference on Learning Representations (ICLR)*.
- [72] Xiaogeng Liu, Zhiyuan Yu, Yizhe Zhang, Ning Zhang, and Chaowei Xiao. 2024. Automatic and Universal Prompt Injection Attacks against Large Language Models. *arXiv preprint arXiv:2403.04957* (2024). doi:10.48550/arxiv.2403.04957
- [73] Carl Macrae. 2022. Learning from the Failure of Autonomous and Intelligent Systems: Accidents, Safety, and Sociotechnical Sources of Risk. *Risk Analysis* 42, 9 (2022), 1999–2025. doi:10.1111/risa.13850
- [74] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. 2024. HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal. *arXiv preprint arXiv:2402.04249*. doi:10.48550/arxiv.2402.04249
- [75] Leo McKee-Reid, Christoph Sträter, María Angélica Martínez, Jon Needham, and Mikita Balesni. 2024. Honesty to Subterfuge: In-Context Reinforcement Learning Can Make Honest Models Reward Hack. *arXiv preprint arXiv:2410.06491* (2024). doi:10.48550/arxiv.2410.06491
- [76] Cameron Mehlman, Joseph Abramov, and Gregory Falco. 2024. Cat-and-Mouse Satellite Dynamics: Divergent Adversarial Reinforcement Learning for Contested Multi-Agent Space Operations. *arXiv preprint arXiv:2409.17443* (2024). doi:10.48550/arxiv.2409.17443
- [77] Alexander Meinke, Bronson Schoen, Jérémy Scheurer, Mikita Balesni, Rusheb Shah, and Marius Hobbhahn. 2024. Frontier Models are Capable of In-context Scheming. *arXiv preprint arXiv:2412.04984* (2024). doi:10.48550/arxiv.2412.04984
- [78] Yuchun Miao, Sen Zhang, Liang Ding, Rong Bao, Lefei Zhang, and Dacheng Tao. 2024. Mitigating Reward Hacking via Information-Theoretic Reward Modeling. *arXiv preprint arXiv:2402.09345* (2024). doi:10.48550/arxiv.2402.09345
- [79] MIT AI Agent Index Team. 2025. *The 2025 AI Agent Index: Documenting Technical and Safety Features of Deployed Agentic AI Systems*. Technical Report. MIT.

- [80] Jesse Mu and Jacob Andreas. 2020. Compositional Explanations of Neurons. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [81] Alexandre Moreira Nascimento, Lucio F. Vismari, C. B. S. T. Molina, Paulo Sérgio Cugnasca, João Batista Camargo, Jorge Rady de Almeida, Rafia Inam, Elena Fersman, M. V. Marquezini, and Alberto Yukinobu Hata. 2020. A Systematic Literature Review About the Impact of Artificial Intelligence on Autonomous Vehicle Safety. *IEEE Transactions on Intelligent Transportation Systems* 21, 12 (2020), 4928–4946. doi:10.1109/tits.2019.2949915
- [82] National Highway Traffic Safety Administration. 2025. NHTSA Opens Probe into Nearly 3 Million Tesla Vehicles Running Full Self-Driving Software. NHTSA Investigation Report.
- [83] National Public Radio. 2025. Waymo Will Recall Software After Its Self-Driving Cars Passed Stopped School Buses. NPR.
- [84] National Transportation Safety Board. 2017. *Collision Between a Car Operating With Automated Vehicle Control Systems and a Tractor-Semitrailer Truck Near Williston, Florida, May 7, 2016*. Highway Accident Report NTSB/HAR-17/02. National Transportation Safety Board, Washington, DC.
- [85] National Transportation Safety Board. 2019. *Assumptions Used in the Safety Assessment Process and the Effects of Multiple Alerts and Indications on Pilot Performance*. Safety Recommendation Report ASR-19-01. National Transportation Safety Board, Washington, DC.
- [86] National Transportation Safety Board. 2019. *Collision Between Vehicle Controlled by Developmental Automated Driving System and Pedestrian, Tempe, Arizona, March 18, 2018*. Highway Accident Report NTSB/HAR-19/03. National Transportation Safety Board, Washington, DC.
- [87] National Transportation Safety Board. 2020. *Collision Between a Sport Utility Vehicle Operating With Partial Driving Automation and a Crash Attenuator, Delray Beach, Florida, March 1, 2019*. Highway Accident Report NTSB/HAR-20/01. National Transportation Safety Board, Washington, DC.
- [88] Gina Neff and Peter Nagy. 2016. Talking to Bots: Symbiotic Agency and the Case of Tay. *International Journal of Communication* 10 (2016), 4915–4931.
- [89] Thanh Nguyen et al. 2025. Empirical Evidence for Alignment Faking in a Small LLM and Prompt-Based Mitigation Techniques. *AAAI Spring Symposium* (2025).
- [90] Laurent Orseau and Stuart Armstrong. 2016. Safely Interruptible Agents. In *Proceedings of the 32nd Conference on Uncertainty in Artificial Intelligence (UAI)*.
- [91] Alvaro Lopez Pellicer, Plamen P. Angelov, and Neeraj Suri. 2025. Securing (Vision-Based) Autonomous Systems: Taxonomy, Challenges, and Defense Mechanisms against Adversarial Threats. *Artificial Intelligence Review* 58 (2025). doi:10.1007/s10462-025-11373-w
- [92] Ram Potham and Max Harms. 2025. Corrigibility as a Singular Target: A Vision for Inherently Reliable Foundation Models. *arXiv preprint arXiv:2506.03056* (2025). doi:10.48550/arxiv.2506.03056
- [93] Pedram Rabiee and Amirsaied Safari. 2023. Safe Exploration in Reinforcement Learning: Training Backup Control Barrier Functions with Zero Training Time Safety Violations. *arXiv preprint arXiv:2312.07828* (2023). doi:10.48550/arxiv.2312.07828
- [94] Ram Rachum, Yonatan Nakar, Bill Tomlinson, Nitay Alon, and Reuth Mirsky. 2024. Emergent Dominance Hierarchies in Reinforcement Learning Agents. *arXiv preprint arXiv:2401.12258* (2024). doi:10.48550/arxiv.2401.12258
- [95] Karolis Ramanauskas and Özgür Şimşek. 2023. Colour versus Shape Goal Misgeneralization in Reinforcement Learning: A Case Study. *arXiv preprint arXiv:2312.03762* (2023). doi:10.48550/arxiv.2312.03762
- [96] Richard Ren, Arunim Agarwal, Mantas Mazeika, Cristina Menghini, Robert Vacareanu, Brad Kenstler, Mick Yang, Isabelle Barrass, Alice Gatti, Xuwang Yin, Eduardo Trevino, Matias Gernalnik, Adam Khoja, Dean Lee, Summer Yue, and Dan Hendrycks. 2025. The MASK Benchmark: Disentangling Honesty From Accuracy in AI Systems. *arXiv preprint arXiv:2503.03750* (2025). doi:10.48550/arxiv.2503.03750
- [97] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. 2023. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. *arXiv preprint arXiv:2309.15817* (2023). doi:10.48550/arxiv.2309.15817
- [98] Karim Abdel Sadek, Matthew Farrugia-Roberts, Usman Anwar, Hannah Erlebach, Christian Schröder de Witt, David Krueger, and Michael Dennis. 2025. Mitigating Goal Misgeneralization via Minimax Regret. *arXiv preprint arXiv:2507.03068* (2025). doi:10.48550/arxiv.2507.03068
- [99] Takami Sato, Ryo Suzuki, Yuki Hayakawa, Kazuma Ikeda, Ozora Sako, Rokuto Nagata, Ryo Yoshida, Qi Alfred Chen, and Kentaro Yoshioka. 2025. On the Realism of LiDAR Spoofing Attacks against Autonomous Driving Vehicle at High Speed and Long Distance. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. doi:10.14722/ndss.2025.230628
- [100] Kelley M. Saylor. 2019. *Defense Primer: U.S. Policy on Lethal Autonomous Weapon Systems*. Technical Report. Congressional Research Service.

- [101] J      Scheurer, Mikita Balesni, and Marius Hobbhahn. 2023. Large Language Models can Strategically Deceive their Users when Put Under Pressure. *arXiv preprint arXiv:2311.07590* (2023).
- [102] Yijia Shao, Tianshi Li, Weiyan Shi, Yanchen Liu, and Diyi Yang. 2024. PrivacyLens: Evaluating Privacy Norm Awareness of Language Models in Action. *arXiv preprint arXiv:2409.00138* (2024). doi:10.48550/arxiv.2409.00138
- [103] Avishag Shapira, P. A. Gandhi, Edan Habler, Oleg Brodt, and Asaf Shabtai. 2025. Mind the Web: The Security of Web Use Agents. *arXiv preprint arXiv:2506.07153* (2025). doi:10.48550/arxiv.2506.07153
- [104] Ibne Farabi Shihab, Sanjeda Akter, and Anuj Sharma. 2025. Detecting and Mitigating Reward Hacking in Reinforcement Learning Systems: A Comprehensive Empirical Study. *arXiv preprint arXiv:2507.05619* (2025). doi:10.48550/arxiv.2507.05619
- [105] Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. 2022. Defining and Characterizing Reward Gaming. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [106] Jack Stilgoe. 2020. *Who's Driving Innovation? New Technologies and the Collaborative State*. Palgrave Macmillan. doi:10.1007/978-3-030-32320-2
- [107] Mark Sullivan. 2025. Replit CEO: What Really Happened When AI Agent Wiped Jason Lemkin's Database. Fast Company.
- [108] Lichao Sun, Yue Huang, Haoran Wang, Siyuan Wu, Qihui Zhang, et al. 2024. TrustLLM: Trustworthiness in Large Language Models. *arXiv preprint arXiv:2401.05561* (2024). doi:10.48550/arxiv.2401.05561
- [109] Bhosale Akshay Tanaji and Sayak Roychowdhury. 2024. A Survey of Cybersecurity Challenges and Mitigation Techniques for Connected and Autonomous Vehicles. *IEEE Transactions on Intelligent Vehicles* (2024). doi:10.1109/tiv.2024.3493938
- [110] Christian Tarsney. 2025. Will Artificial Agents Pursue Power by Default? *arXiv preprint arXiv:2506.06352* (2025). doi:10.48550/arxiv.2506.06352
- [111] Alexander Matt Turner, Logan Smith, Rohin Shah, Andrew Critch, and Prasad Tadepalli. 2021. Optimal Policies Tend To Seek Power. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [112] Alexander Matt Turner and Prasad Tadepalli. 2022. Parametrically Retargetable Decision-Makers Tend To Seek Power. In *Advances in Neural Information Processing Systems (NeurIPS)*. doi:10.48550/arxiv.2206.13477
- [113] Akifumi Wachi, Wataru Hashimoto, Xun Shen, and Kazumune Hashimoto. 2023. Safe Exploration in Reinforcement Learning: A Generalized Formulation and Algorithms. In *Advances in Neural Information Processing Systems (NeurIPS)*. doi:10.48550/arxiv.2310.03225
- [114] Akifumi Wachi, Xun Shen, and Yanan Sui. 2024. A Survey of Constraint Formulations in Safe Reinforcement Learning. In *Proceedings of the 33rd International Joint Conference on Artificial Intelligence (IJCAI)*. doi:10.24963/ijcai.2024/913
- [115] Yifan Wang et al. 2025. Agentic AI Security: Threats, Defenses, Evaluation, and Open Challenges. *arXiv preprint arXiv:2510.23883* (2025). doi:10.48550/arxiv.2510.23883
- [116] Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. 2023. Jailbroken: How Does LLM Safety Training Fail?. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [117] Jeffrey Wen et al. 2025. AgentMisalignment: How LLMs Could Be Insider Threats. *arXiv preprint arXiv:2506.04018* (2025). doi:10.48550/arxiv.2506.04018 Under review at ICLR 2026.
- [118] Marty J. Wolf, Keith Miller, and Frances Grodzinsky. 2017. Why We Should Have Seen That Coming: Comments on Microsoft's Tay "Experiment," and Wider Implications. In *SIGCAS Computers and Society*. doi:10.1145/3144592.3144598
- [119] Nathan Wood. 2023. Autonomous Weapon Systems and Responsibility Gaps: A Taxonomy. *Ethics and Information Technology* 25, 2 (2023). doi:10.1007/s10676-023-09690-1
- [120] Zeyu Wu et al. 2025. LLMs Learn to Deceive Unintentionally: Emergent Misalignment in Dishonesty from Misaligned Samples to Biased Human-AI Interactions. *arXiv preprint arXiv:2510.08211* (2025). doi:10.48550/arxiv.2510.08211
- [121] Chejian Xu, Wenhao Ding, Weijie Lyu, Zuxin Liu, Shuai Wang, Yi He, Hanjiang Hu, Ding Zhao, and Bo Li. 2022. SafeBench: A Benchmarking Platform for Safety Evaluation of Autonomous Vehicles. In *Advances in Neural Information Processing Systems (NeurIPS)*. doi:10.48550/arxiv.2206.09682
- [122] Yuqing Yang, Ethan Chern, Xipeng Qiu, Graham Neubig, and Pengfei Liu. 2023. Alignment for Honesty. *arXiv preprint arXiv:2312.07000* (2023). doi:10.48550/arxiv.2312.07000
- [123] Kaiyuan Zhang, Zian Su, Pin-Yu Chen, Elisa Bertino, Xiangyu Zhang, and Ninghui Li. 2025. LLM Agents Should Employ Security Principles. *arXiv preprint arXiv:2505.24019* (2025). doi:10.48550/arxiv.2505.24019
- [124] Zhixin Zhang, Shiyao Cui, Yida Lu, Jingzhuo Zhou, Junxiao Yang, Hongning Wang, and Minlie Huang. 2024. Agent-SafetyBench: Evaluating the Safety of LLM Agents. *arXiv preprint arXiv:2412.14470* (2024). doi:10.48550/arxiv.2412.14470
- [125] Simon Zhuang and Dylan Hadfield-Menell. 2021. Consequences of Misaligned AI. *Advances in Neural Information Processing Systems (NeurIPS)* (2021).